

DAT A LINK LAYER DESIGN ISSUES

The data link layer uses the services of the physical layer below it to send and receive bits over (possibly unreliable) communication channels that may lose data.

It has a number of functions, including:

1. Providing a well-defined service interface to the network layer
2. Framing sequences of bytes as self-contained segments.
3. Detecting and correcting transmission errors.
4. Regulating the flow of data so that slow receivers are not swamped by fast senders.

To accomplish these goals, the data link layer takes the packets it gets from the network layer and encapsulates them into frames for transmission. Each frame contains a frame header, a payload field for holding the packet, and a frame trailer, as illustrated in below Fig

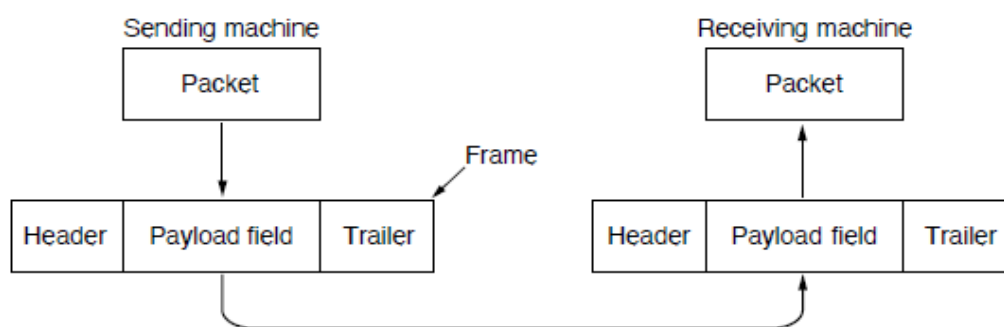


Figure 3-1. Relationship between packets and frames.

3.1.1 Services Provided to the Network Layer

The function of the data link layer is to provide services to the network layer. The principal service of the link layer is transferring data from the network layer on the source machine to the network layer on the destination machine. On the source machine is an entity, call it a process, in the network layer that passes packets to the data link layer for transmission to the destination. The job of the data link layer is to transmit the data to the destination machine so they can be handed over to the network layer there, as shown in Fig. 3-2(a). The actual transmission follows the path of Fig. 3-2(b), but it is easier to think in terms of two data link layer processes communicating using a data link protocol. For this reason, we will implicitly use the model of Fig. 3-2(a) throughout this chapter.

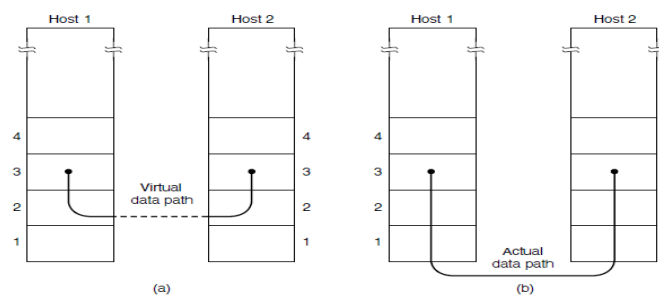


Figure 3-2. (a) Virtual communication. (b) Actual communication.

The data link layer can be designed to offer various services. The actual services that are offered vary from protocol to protocol. Three reasonable possibilities that we will consider in turn are:

1. Unacknowledged connectionless service.
2. Acknowledged connectionless service.
3. Acknowledged connection-oriented service.

Unacknowledged connectionless service consists of having the source machine send independent frames to the destination machine without having the destination machine acknowledge them.

Ethernet is a good example of a data link layer that provides this class of service. No logical connection is established beforehand or released afterward. If a frame is lost due to noise on the line, no attempt is made to detect the loss or recover from it in the data link layer. The next step up in terms of reliability is acknowledged connectionless service. When this service is offered, there are still no logical connections used, but each frame sent is individually acknowledged. In this way, the sender knows whether a frame has arrived correctly or been lost. If it has not arrived within a specified time interval, it can be sent again.

It is perhaps worth emphasizing that providing acknowledgements in the data link layer is just an optimization. It is never a requirement. The network layer can always send a packet and wait for it to be acknowledged by its peer on the remote machine. If the acknowledgement is not received before a retransmission timer expires, the sender can just send the entire message again. The trouble with this strategy is that it can be inefficient. Links frequently have a strict maximum frame length imposed by the hardware, and known propagation delays. The network layer does not know these parameters. It might send a large packet that is broken up into, say, ten frames, of which two are lost on average. It would then take a very long time for the packet to get through. Instead, if individual frames are acknowledged and retransmitted, then errors can be corrected more directly and more quickly. On reliable channels, such as fiber, the overhead of a heavyweight data link layer protocol may be unnecessary, but on (inherently unreliable) wireless channels the overhead is often worth the cost.

3.1.2 Framing

To provide service to the network layer, the data link layer must use the service provided to it by the physical layer. The physical layer accepts a raw bit stream and attempts to deliver it to the destination. If the channel is noisy, as it is for most wireless and some wired links, the physical layer will add some redundancy to its signals to reduce the bit error rate to a tolerable level. However, the bit stream received by the data link layer is not guaranteed to be error-free. Some bits may have different values, and the number of bits received may be less than, equal to, or more than the number of bits transmitted. It is up to the data link layer to detect and, if necessary, correct errors.

The usual approach is for the data link layer to break up the bit stream into discrete frames, compute a short token called a checksum for each frame, and include the checksum in the frame when it is transmitted. (Checksum algorithms will be discussed later in this chapter.) When a frame arrives at the destination, the receiver recomputes the checksum based on the received frame. If the newly computed checksum is different from the one contained in the frame, the data link layer knows that an error has occurred and takes steps to deal with it (e.g., discarding the bad frame and possibly also sending back an error report).

Breaking up the bit stream into frames is more difficult than it at first appears. A good design must make it easy for a receiver to find the start of new frames while using little of the channel bandwidth. We will look at four methods:

1. Byte count.
2. Flag bytes with byte stuffing.
3. Flag bits with bit stuffing.
4. Physical layer coding violations.

The first framing method uses a field in the header to specify the number of bytes in the frame. When the data link layer at the destination sees the byte count, it knows how many bytes follow and hence where the end of the frame is. This technique is shown in Fig. 3-3(a) for four small example frames of sizes 5, 5, 8, and 8 bytes, respectively. The trouble with this algorithm is that the count can be garbled by a transmission error. For example, if the byte count of 5 in the second frame of Fig. 3-3(b) becomes a 7 due to a single bit flip, the destination will get out of synchronization. It will then be unable to locate the correct start of the next frame. Even if the checksum is incorrect so the destination knows that the frame is bad, it still has no way of telling where the next frame starts.

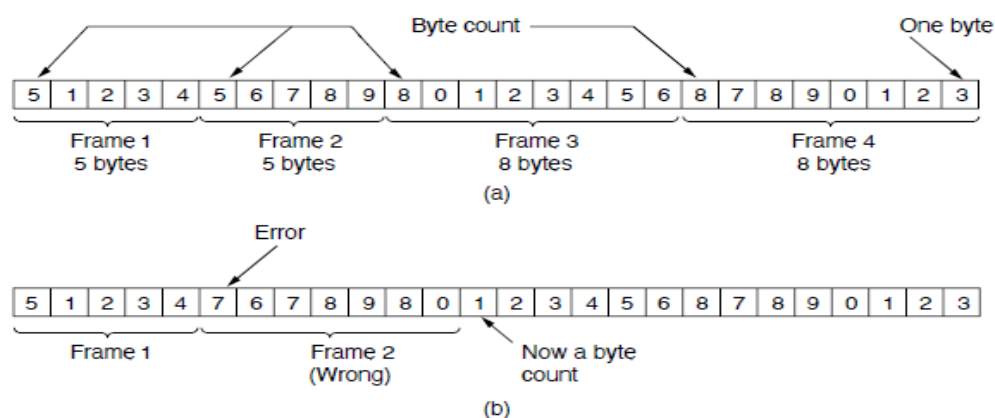


Figure 3-3. A byte stream. (a) Without errors. (b) With one error.

Error Detection and Correction with Parity

Overview

Error detection and correction are techniques used to ensure the accuracy of data transmitted over a communication channel or stored in memory. One of the simplest methods for error detection is the **parity bit**. The parity bit helps detect errors in the transmission of binary data by ensuring the number of 1's (or 0's) in a data unit is either even or odd.

1. What is Parity?

Parity is a method to detect errors in binary data. A **parity bit** is added to a set of data bits to make the total number of 1's either even or odd. The parity can be:

- **Even Parity:** The number of 1's in the data, including the parity bit, is even.
- **Odd Parity:** The number of 1's in the data, including the parity bit, is odd.

The parity bit itself is set to either 0 or 1, depending on the desired parity.

2. Parity Bit for Error Detection

The idea behind the parity bit is simple:

- **Sender:** The sender computes the parity bit for the data being sent.
- **Receiver:** The receiver checks the parity of the received data, including the parity bit, and compares it with the expected parity (even or odd).

If the parity doesn't match the expected value, an error has occurred during transmission, and the receiver can ask for the data to be sent again.

Example of Even Parity

Let's say we are transmitting the 4-bit data: 1101

- **Step 1:** Count the number of 1's in the data.
 - The number of 1's in 1101 is 3 (which is odd).
- **Step 2:** Add a parity bit to make the total number of 1's even.
 - Since we want even parity, we need to add a 1 (because 3 is odd, and adding a 1 will make the count of 1's even).
- **Step 3:** The transmitted data will be 11011 (data + parity bit).

Example of Odd Parity

Now, consider we are transmitting the 4-bit data: 1010

- **Step 1:** Count the number of 1's in the data.
 - The number of 1's in 1010 is 2 (which is even).
 - **Step 2:** Add a parity bit to make the total number of 1's odd.
 - Since we want odd parity, we need to add a 1 (because 2 is even, and adding a 1 will make the count of 1's odd).
 - **Step 3:** The transmitted data will be 10101 (data + parity bit).
-

3. Parity for Error Detection

Once the data with the parity bit is transmitted to the receiver, the receiver can verify whether the data has been transmitted correctly.

Receiver's Role:

The receiver will:

1. **Check the parity of the received data** (including the parity bit).
2. If the number of 1's is not consistent with the expected parity (even or odd), an error is detected.

Example of Error Detection

- **Receiver receives:** 11011 (data with parity bit).
- **Receiver checks:** Number of 1's in 11011 is 4 (which is even), so the data is correct if even parity is expected.

Now, let's say there was a bit error during transmission, and the receiver gets 11111 (one bit has changed):

- **Receiver checks:** Number of 1's in 11111 is 5 (which is odd), but we were expecting even parity. Thus, the receiver detects an error.
-

4. Error Correction with Parity (Basic Error Correction)

While parity can detect errors, it **cannot correct errors** on its own. It can only indicate if an error has occurred.

Correcting Single Bit Errors with Parity:

In some systems, **error-correcting codes** (such as Hamming codes) can be used in combination with parity to not only detect errors but also correct them. Parity is often used as a **part of a larger error-correction scheme**.

For example, if a system uses both a **parity bit** and a **Hamming code** (which adds redundancy to the data and can correct single-bit errors), the parity bit can help identify errors and the Hamming code can correct them.

5. Parity in Practical Use

1. Parity in Memory Systems

In memory systems, parity is often used to detect errors in the data stored or transmitted between the CPU and memory. For instance, a single bit parity is added to each byte in a memory system.

2. Parity in Network Communication

When data is transmitted over a network, the parity bit is added to the packet of data. The receiver checks the parity bit to determine if an error has occurred in the transmission. If an error is detected, the receiver requests a retransmission of the data.

3. Parity in Storage Devices

Many hard drives, CDs, and other storage devices use parity bits to detect errors in stored data, ensuring the integrity of stored files.

Diagram of Parity Bit Error Detection:

```
Data to Transmit:      1101
Parity Bit:             1      (Makes the total number of 1's even)
Transmitted Data:      11011

Receiver checks:
Received Data:          11011
Number of 1's:          4 (Even) - No Error (if even parity expected)
```

If one bit is corrupted in transmission:

```
Transmitted Data:      11011
Receiver receives:     11111
Receiver checks:
Number of 1's:          5 (Odd) - Error detected
```

6. Limitations of Parity

- **Error Detection:** Parity can only detect an **odd number of errors**. It cannot detect **even numbers of errors**. If two bits are flipped, the parity will still be correct, and the error will go undetected.
- **Error Correction:** Parity alone cannot correct errors. To correct errors, more sophisticated error-correcting codes (like Hamming Code) are needed.

Conclusion

Parity is a simple yet effective method for error detection, ensuring that the transmitted data is valid. It is often used in combination with other error-correction techniques in more

complex systems. Although it's not sufficient for correcting errors on its own, it provides a fast and simple way to detect single-bit errors in communication or storage systems.

Certainly! Here's a **detailed explanation of Error Detection and Correction using CRC (Cyclic Redundancy Check)**, including diagrams and examples.

Error Detection and Correction Using CRC (Cyclic Redundancy Check)

1. What is CRC?

CRC (Cyclic Redundancy Check) is a powerful method used for **error detection** in digital networks and storage devices. It helps to detect accidental changes/errors in raw data by treating the data as a polynomial and performing polynomial division with a predetermined generator polynomial.

- CRC is widely used because it can detect common types of errors (e.g., burst errors) more reliably than parity bits or checksums.
- It is primarily an **error detection technique, not error correction** by itself.
- If an error is detected, the receiver can request retransmission.

2. How Does CRC Work?

- The data to be transmitted is treated as a binary number (polynomial).
- A **generator polynomial (G(x))** is agreed upon by both sender and receiver.
- The sender appends extra bits, called **CRC bits (checksum)**, to the original data.
- The receiver uses the same generator polynomial to check if the received data (including CRC) divides exactly by the generator polynomial.
- If not, an error is detected.

3. Mathematical Background

- Data is represented as a polynomial $D(x)$.
 - Generator polynomial is $G(x)$.
 - The sender calculates the remainder $R(x)$ when $D(x) \times x^r$ is divided by $G(x)$, where r is the degree of $G(x)$.
 - The transmitted message is $T(x) = D(x) \times x^r + R(x)$.
 - The receiver divides $T(x)$ by $G(x)$; if remainder is zero, no error detected.
-

4. Steps for CRC Calculation

Suppose:

- Original data: DD
 - Generator polynomial length (degree): rr
 - Append rr zero bits to data.
 - Divide augmented data by $G(x)$ using modulo-2 division (XOR operations).
 - Remainder is CRC bits.
 - Append remainder to original data for transmission.
-

5. Example

Let's take a simple example:

- Data bits (D): **11010011101100**
 - Generator polynomial (G): **1011** (degree 3, so append 3 zeros)
-

Step 1: Append 3 zeros to data:

11010011101100 000

Step 2: Divide augmented data by generator using modulo-2 division:

- Use XOR instead of subtraction.
 - Perform division similar to long division but XOR bits when divisor fits under the current bits.
-

Step 3: The remainder after division is CRC bits (3 bits since degree is 3).

Step 4: Append CRC remainder to original data:

Transmitted Data = Original Data + CRC bits

6. Diagram of CRC Process

Sender Side:

Original Data: 11010011101100
11010011101100 + CRC

Receiver Side:

Received Data:

Append zeros:	11010011101100 000	Divide by G(x):	Check
remainder			
Divide by G(x):	Calculate remainder	If remainder != 0,	error
detected			
Append remainder:	Original Data + CRC		
Transmit data:	11010011101100 + CRC		

7. Modulo-2 Division Example

Let's do the modulo-2 division (XOR division) for part of the example:

- Data (augmented): 11010011101100000 (14 bits + 3 zeros = 17 bits)
- Generator: 1011 (4 bits)

Division Steps:

- Take the first 4 bits of data: 1101
 - If the first bit is 1, XOR with 1011, else do nothing.
 - Continue with next bits until all bits are processed.
 - The remainder after full division is the CRC.
-

8. Error Detection

- The receiver performs the same division on the received data.
 - If the remainder is zero, no error.
 - If remainder is non-zero, error detected.
-

9. Error Correction

- CRC itself does **not correct errors**.
 - It only detects errors.
 - If error detected, data is requested to be resent.
 - For error correction, additional methods like Forward Error Correction (FEC) codes (Hamming code, Reed-Solomon) are used.
-

10. Advantages of CRC

- Detects single-bit errors, double-bit errors, odd number of errors.
 - Detects burst errors up to the length of the generator polynomial.
 - Simple hardware implementation (XOR gates).
 - Widely used in Ethernet, USB, storage devices.
-

11. Summary Table

Feature	Description
Purpose	Error Detection
Method	Polynomial division modulo-2 (XOR operations)
Data Representation	Binary polynomial
Generator Polynomial	Predetermined agreed polynomial
Error Detection	Detects burst errors and multiple bit errors
Error Correction	No (needs other methods)
Usage	Network protocols, storage devices

Illustration of CRC Process

Data bits: 11010011101100
Append 3 zeros: 11010011101100000
Generator: 1011

Division (XOR) steps:

1101 (data segment) XOR 1011 (generator)
= 0110

Bring down next bit, repeat until all bits processed

Remainder (CRC bits): 100

Transmit: 11010011101100 + 100

Great! Let's go through the **step-by-step modulo-2 division** (XOR division) for the CRC example in detail.

Step-by-Step Modulo-2 Division for CRC

Given:

- **Data bits (D):** 11010011101100 (14 bits)
- **Generator (G):** 1011 (4 bits)
- Append 3 zeros (degree of G is 3) to data → 11010011101100 000 (17 bits)

Objective:

Divide 11010011101100000 by 1011 using modulo-2 division to find the **remainder (CRC bits)**.

Modulo-2 Division Rules:

- Division is similar to long division but instead of subtraction, use XOR.
 - XOR rule:
 - $1 \text{ XOR } 1 = 0$
 - $0 \text{ XOR } 0 = 0$
 - $1 \text{ XOR } 0 = 1$
 - $0 \text{ XOR } 1 = 1$
 - Align the divisor under the leftmost '1' bit of the dividend segment.
 - Perform XOR between divisor and that segment.
 - Drop down the next bit of the dividend.
 - Repeat until all bits are processed.
 - The remainder will be the last few bits that are left after you cannot divide further (length = degree of divisor = 3).
-

Step 1: Setup

Dividend (data with appended zeros):

11010011101100000

(17 bits)

Divisor (generator):

1011

(4 bits)

Step 2: Perform division (XOR):

We will work on 4-bit segments since divisor length is 4.

Step	Current bits to divide	XOR with divisor?	Result after XOR	Next bits to bring down
1	1101	Yes (first bit 1)	$1101 \text{ XOR } 1011 =$	Bring down next bit: 0 →

Step	Current bits to divide	XOR with divisor?	Result after XOR	Next bits to bring down
			0110	01100
2	1100 (leftmost bit is 0, so move to next)	No (first bit 0)	Keep as 1100	Drop leftmost bit → 100
3	1001 (shifted)	Yes (first bit 1)	1001 XOR 1011 = 0010	Bring down next bit: 1 → 00101
4	0101 (leftmost bit 0, move on)	No	Keep as 0101	Drop leftmost bit → 101
5	1011	Yes	1011 XOR 1011 = 0000	Bring down next bit: 1 → 00001
6	0001 (leftmost bit 0, move on)	No	Keep as 0001	Drop leftmost bit → 001
7	0011 (leftmost bit 0, move on)	No	Keep as 0011	Drop leftmost bit → 011
8	0110 (leftmost bit 0, move on)	No	Keep as 0110	Drop leftmost bit → 110
9	1100	Yes	1100 XOR 1011 = 0111	Bring down next bit: 0 → 01110
10	1110	Yes	1110 XOR 1011 = 0101	Bring down next bit: 0 → 01010
11	1010	Yes	1010 XOR 1011 = 0001	Bring down next bit: 0 → 00010
12	0010 (leftmost bit 0)	No	Keep as 0010	Drop leftmost bit → 010

Step 3: Remainder after last division

The last bits left (length = 3) are: 010

Step 4: Final CRC bits = 010

Step 5: Transmitted Frame

Original data:
11010011101100

Append CRC:
010

Transmitted frame:
11010011101100 010

Summary Table (with XOR operations)

Step	Dividend Segment	Divisor (G)	XOR Result	Comments
1	1101	1011	0110	XOR, bring down next bit 0 → 01100
2	1100	-	-	Leading bit 0, shift left → 100
3	1001	1011	0010	XOR, bring down next bit 1 → 00101
4	0101	-	-	Leading bit 0, shift left → 101
5	1011	1011	0000	XOR, bring down next bit 1 → 00001
6	0001	-	-	Leading bit 0, shift left → 001
7	0011	-	-	Leading bit 0, shift left → 011
8	0110	-	-	Leading bit 0, shift left → 110
9	1100	1011	0111	XOR, bring down next bit 0 → 01110
10	1110	1011	0101	XOR, bring down next bit 0 → 01010
11	1010	1011	0001	XOR, bring down next bit 0 → 00010
12	0010	-	-	Final remainder: 010

Final Notes:

- The **remainder (CRC bits)** is what gets appended to the original data.
- The **receiver** performs the same modulo-2 division on the entire received frame (data + CRC).
- If the remainder is zero, data is considered **error-free**.
- If remainder is non-zero, an error is detected.

Certainly! Here's a **detailed explanation of Error Detection and Correction using Hamming Code**, complete with diagrams and examples.

Error Detection and Correction Using Hamming Code

1. What is Hamming Code?

Hamming Code is an error-detecting and error-correcting code invented by Richard Hamming. It is widely used for detecting and correcting **single-bit errors** in data transmission or storage.

- It adds **redundant parity bits** at specific positions in the data.
 - Can detect and **correct single-bit errors**.
 - Can **detect (but not correct) two-bit errors**.
-

2. Basic Idea

- Original data bits are mixed with parity bits at positions which are powers of two: 1, 2, 4, 8, 16, etc.
 - Each parity bit checks bits at positions that have a 1 in the parity bit's binary position.
 - By recalculating parity bits at the receiver and comparing with received parity bits, you can locate exactly which bit (if any) is erroneous.
-

3. Positions of Parity Bits

For data bits length m and parity bits length r , the total length $n = m + r$ must satisfy:

$$2^r \geq m + r + 1$$

The parity bits are placed at positions:

$$1, 2, 4, 8, \dots, 2^{r-1}$$

4. Example: Hamming(7,4) Code

- **4 data bits** ($m = 4$)
 - **3 parity bits** ($r = 3$)
 - Total bits: 7 ($n = 7$)
 - Parity bits at positions: 1, 2, 4
 - Data bits at positions: 3, 5, 6, 7
-

5. Step-by-Step Encoding

Suppose data bits to send are:

$$d_3=1, d_5=0, d_6=1, d_7=1 \quad d_3 = 1, \quad d_5 = 0, \quad d_6 = 1, \quad d_7 = 1$$

Position	1	2	3	4	5	6	7
----------	---	---	---	---	---	---	---

Bit type	P1	P2	D3	P4	D5	D6	D7
----------	----	----	----	----	----	----	----

Value	?	?	1	?	0	1	1
-------	---	---	---	---	---	---	---

6. Parity Bit Calculation

Each parity bit covers certain bit positions (based on binary representation):

- **P1** covers bits with a 1 in bit 1 (LSB) of position index: positions 1, 3, 5, 7
- **P2** covers bits with a 1 in bit 2 of position index: positions 2, 3, 6, 7
- **P4** covers bits with a 1 in bit 4 of position index: positions 4, 5, 6, 7

Calculate parity bits to make the parity even (even parity):

-
- **P1 parity bit:** bits 1, 3, 5, 7

$$P1 \oplus d_3 \oplus d_5 \oplus d_7 = 0 \Rightarrow P1 \oplus 1 \oplus 0 \oplus 1 = 0 \quad P1 \oplus d_3 \oplus d_5 \oplus d_7 = 0 \quad \Rightarrow \quad P1 \oplus 1 \oplus 0 \oplus 1 = 0 \quad P1 \oplus 0 = 0 \Rightarrow P1 = 0$$

-
- **P2 parity bit:** bits 2, 3, 6, 7

$$P2 \oplus d_3 \oplus d_6 \oplus d_7 = 0 \Rightarrow P2 \oplus 1 \oplus 1 \oplus 1 = 0 \quad P2 \oplus d_3 \oplus d_6 \oplus d_7 = 0 \quad \Rightarrow \quad P2 \oplus 1 \oplus 1 \oplus 1 = 0 \quad P2 \oplus 1 = 0 \Rightarrow P2 = 1$$

-
- **P4 parity bit:** bits 4, 5, 6, 7

$$P4 \oplus d_5 \oplus d_6 \oplus d_7 = 0 \Rightarrow P4 \oplus 0 \oplus 1 \oplus 1 = 0 \quad P4 \oplus d_5 \oplus d_6 \oplus d_7 = 0 \quad \Rightarrow \quad P4 \oplus 0 \oplus 1 \oplus 1 = 0 \quad P4 \oplus 0 = 0 \Rightarrow P4 = 0$$

7. Encoded 7-bit Hamming Code

Position 1 2 3 4 5 6 7

Bits 0 1 1 0 0 1 1

Transmitted Code: **0 1 1 0 0 1 1**

8. Error Detection and Correction

Suppose the received code is:

0 1 0 0 1 1 (bit 3 flipped from 1 to 0)

Step 1: Recalculate parity bits (Syndrome bits)

- $S_1 = P_1 \oplus d_3 \oplus d_5 \oplus d_7$
 $= 0 \oplus 0 \oplus 0 \oplus 1 = 1 = 0 \oplus 0 \oplus 0 \oplus 1 = 1$
 - $S_2 = P_2 \oplus d_3 \oplus d_6 \oplus d_7$
 $= 1 \oplus 0 \oplus 1 \oplus 1 = 1 = 1 \oplus 0 \oplus 1 \oplus 1 = 1$
 - $S_4 = P_4 \oplus d_5 \oplus d_6 \oplus d_7$
 $= 0 \oplus 0 \oplus 1 \oplus 1 = 0 = 0 \oplus 0 \oplus 1 \oplus 1 = 0$
-

Step 2: Find Error Position

The error position is found by combining syndrome bits ($S_4 S_2 S_1$) in binary:

$$S = S_4 S_2 S_1 = 011 = 3 \quad S = S_4 S_2 S_1 = 0 1 1 = 3$$

This indicates the error is in bit 3.

Step 3: Correct Error

Flip bit 3 back from 0 to 1:

Received corrected code:

0 1 1 0 0 1 1 (same as sent)

9. Diagram

Position: 1 2 3 4 5 6 7
Bits: P1 P2 D3 P4 D5 D6 D7

P1 checks bits: 1, 3, 5, 7

P2 checks bits: 2, 3, 6, 7

P4 checks bits: 4, 5, 6, 7

At receiver:

Calculate syndrome bits (S1, S2, S4)

Error position = binary(S4 S2 S1)

If 0, no error; else flip bit at error position.

10. Summary

Feature	Description
Code Type	Error Detection & Correction
Capable of correcting	Single-bit errors
Can detect	Two-bit errors
Parity Bits Positions	Powers of 2 (1, 2, 4, 8, ...)
Total Bits = Data + Parity	$n=m+rn = m + r$
Example	Hamming(7,4)

11. Additional Notes

- For longer data, more parity bits are added.
 - The principle remains the same.
 - Widely used in computer memory (ECC RAM), communications.
-

Awesome! Let's walk through a **complete step-by-step example of Hamming(7,4) code encoding, transmission (with error), detection, and correction**, with detailed explanation and diagrams.

Step-by-Step Example of Hamming Code (7,4)

Given:

- 4-bit data to transmit: **1011**
-

Step 1: Place Data Bits and Parity Bits

Positions 1, 2, 4 are parity bits (P1, P2, P4).
Positions 3, 5, 6, 7 are data bits.

Position **1** **2** **3** **4** **5** **6** **7**

Bit Type P1 P2 D3 P4 D5 D6 D7

Value ? ? 1 ? 0 1 1

Data bits are assigned as follows:

- $D3 = 1D_3 = 1$
 - $D5 = 0D_5 = 0$
 - $D6 = 1D_6 = 1$
 - $D7 = 1D_7 = 1$
-

Step 2: Calculate Parity Bits (Even Parity)

Recall which bits each parity bit covers:

- P1 (position 1):** bits 1, 3, 5, 7
 - P2 (position 2):** bits 2, 3, 6, 7
 - P4 (position 4):** bits 4, 5, 6, 7
-

Calculate P1:

$$P1 \oplus D3 \oplus D5 \oplus D7 = 0 \quad P1 \oplus 1 \oplus 0 \oplus 1 = 0 \quad P1 \oplus 1 \oplus 0 \oplus 1 = 0 \quad P1 \oplus 0 = 0 \implies P1 = 0$$

Calculate P2:

$$P2 \oplus D3 \oplus D6 \oplus D7 = 0 \quad P2 \oplus 1 \oplus 1 \oplus 1 = 0 \quad P2 \oplus 1 \oplus 1 \oplus 1 = 0 \quad P2 \oplus 1 = 0 \implies P2 = 1$$

Calculate P4:

$$P4 \oplus D5 \oplus D6 \oplus D7 = 0 \quad P4 \oplus 0 \oplus 1 \oplus 1 = 0 \quad P4 \oplus 0 \oplus 1 \oplus 1 = 0 \quad P4 \oplus 0 = 0 \Rightarrow P4 = 0$$

Step 3: Final 7-bit Code to Transmit

Position 1 2 3 4 5 6 7

Bits 0 1 1 0 0 1 1

Transmitted Code: 0 1 1 0 0 1 1

Step 4: Simulate Transmission Error

Suppose bit 6 is flipped from 1 $\rightarrow$ 0 during transmission.

Received code: 0 1 1 0 0 0 1

Step 5: Error Detection at Receiver (Calculate Syndrome)

Calculate syndrome bits S_1, S_2, S_4 :

- $S_1 = P1 \oplus D3 \oplus D5 \oplus D7 = 0 \oplus 1 \oplus 0 \oplus 1 = 0$

- $S_2 = P2 \oplus D3 \oplus D6 \oplus D7 = 1 \oplus 1 \oplus 0 \oplus 1 = 1$

- $S_4 = P4 \oplus D5 \oplus D6 \oplus D7 = 0 \oplus 0 \oplus 0 \oplus 1 = 1$

Step 6: Determine Error Position

Combine syndrome bits as binary number:

$$S = S_4 S_2 S_1 = 110_2 = 6_{10} \quad S = S_4 S_2 S_1 = 110_2 = 6_{10}$$

Error detected in bit position **6**.

Step 7: Correct the Error

Flip bit 6 back from 0 $\rightarrow$ 1:

Corrected code:

0 1 1 0 0 1 1 (original transmitted code)

Step 8: Extract Original Data

Data bits at positions 3, 5, 6, 7:

- $D_3 = 1D_3 = 1$
- $D_5 = 0D_5 = 0$
- $D_6 = 1D_6 = 1$
- $D_7 = 1D_7 = 1$

Original data recovered: **1011**

Summary Diagram

Position:	1	2	3	4	5	6	7
Bit Type:	P1	P2	D3	P4	D5	D6	D7
Send:	0	1	1	0	0	1	1
Error:						Bit 6 flipped to 0	
Receive:	0	1	1	0	0	0	1

Calculate Syndrome:

$$S_1 = P_1 \oplus D_3 \oplus D_5 \oplus D_7 = 0 \oplus 1 \oplus 0 \oplus 1 = 0$$

$$S_2 = P_2 \oplus D_3 \oplus D_6 \oplus D_7 = 1 \oplus 1 \oplus 0 \oplus 1 = 1$$

$$S_4 = P_4 \oplus D_5 \oplus D_6 \oplus D_7 = 0 \oplus 0 \oplus 0 \oplus 1 = 1$$

$$\text{Error position} = \text{binary}(S_4 S_2 S_1) = 110_2 = 6$$

Correct bit 6 by flipping 0 $\rightarrow$ 1

Extract original data bits: 1011

Absolutely! Let me give you a detailed explanation of the **Stop-and-Wait ARQ Protocol**, one of the fundamental Automatic Repeat reQuest (ARQ) protocols used in data communication to ensure reliable transmission.

Stop-and-Wait ARQ Protocol — Detailed Explanation

What is ARQ?

ARQ (Automatic Repeat reQuest) protocols are error-control protocols used in data communication systems. Their main goal is to ensure reliable data transfer over an unreliable communication channel by detecting errors and retransmitting lost or corrupted data frames.

Overview of Stop-and-Wait ARQ

Stop-and-Wait ARQ is the simplest ARQ protocol. It works on the principle that the sender sends one frame (packet) at a time and then waits for an acknowledgment (ACK) from the receiver before sending the next frame.

How Does Stop-and-Wait ARQ Work?

Here is the step-by-step operation:

1. **Sender sends a frame:** The sender transmits a single data frame to the receiver.
 2. **Sender waits for ACK:** After sending the frame, the sender stops and waits for an acknowledgment (ACK) from the receiver.
 3. **Receiver receives frame:**
 - If the frame is **received correctly** (no errors), the receiver sends back an ACK to the sender.
 - If the frame is **corrupted** or lost, the receiver does not send an ACK (or sends a negative acknowledgment NAK, depending on implementation).
 4. **Sender receives ACK:**
 - Upon receiving the ACK, the sender proceeds to send the next frame.
 - If no ACK is received within a timeout period, the sender **retransmits** the same frame, assuming it was lost or corrupted.
-

Important Components

- **Frame Sequence Numbers:**
Since only one frame is sent at a time, the sequence number is often just 1 bit (e.g., 0 and 1 alternately) to differentiate between new frames and retransmissions.
 - **Timeout Timer:**
The sender sets a timer after sending each frame. If the timer expires without receiving an ACK, it retransmits the frame.
-

Why Use Sequence Numbers?

Because the sender retransmits the frame after a timeout, the receiver needs to know if the received frame is a **new frame** or a **retransmission of the previous frame** to avoid duplicate data. The 1-bit sequence number alternates between 0 and 1:

- Frame 0 sent → Receiver ACK 0 → Sender sends Frame 1 → Receiver ACK 1 → and so on.
-

Example Scenario

Step	Action	Sender	Receiver
1	Send Frame 0	Sends Frame 0	Receives Frame 0
2	Frame received correctly	Waits for ACK	Sends ACK 0
3	ACK received	Sends Frame 1	Waits for Frame 1
4	Frame lost or corrupted	Waits for ACK (timeout starts)	Frame lost
5	Timeout expires	Retransmits Frame 1	Receives Frame 1
6	Frame received correctly	Waits for ACK	Sends ACK 1
7	ACK received	Sends next frame	...

Advantages of Stop-and-Wait ARQ

- **Simple and easy to implement.**
 - Ensures reliable transmission by confirming each frame before sending the next.
 - Sequence numbers prevent duplication.
-

Disadvantages of Stop-and-Wait ARQ

- **Inefficient for high-latency or high-bandwidth networks**, because the sender is idle (waiting) during the round-trip time (RTT).
 - Throughput is low since only one frame is sent at a time.
 - Poor utilization of the communication channel, especially in long-delay links (e.g., satellite communication).
-

Performance Metrics

Efficiency (η):

$$\eta = \frac{T_{\text{frame}}}{T_{\text{frame}} + 2 \times T_{\text{prop}} + T_{\text{ACK}}}$$

Where:

- T_{frame} = Time to send one frame.
- T_{prop} = Propagation delay (one way).
- T_{ACK} = Time to send ACK (usually small).

Since the sender waits for the ACK, the efficiency decreases drastically when propagation delay is large compared to transmission time.

Figure 11.10 *Stop-and-Wait protocol*

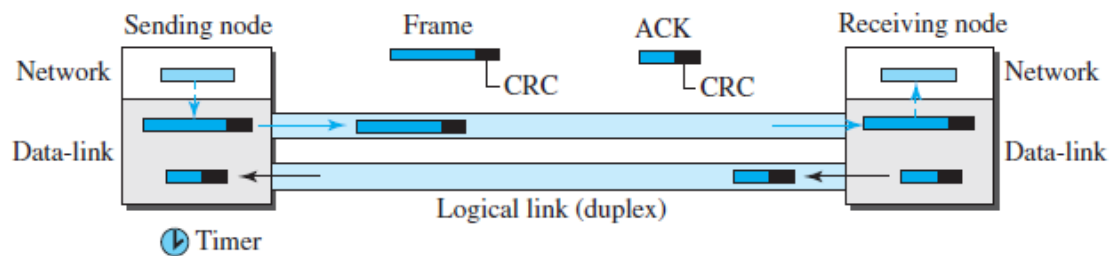
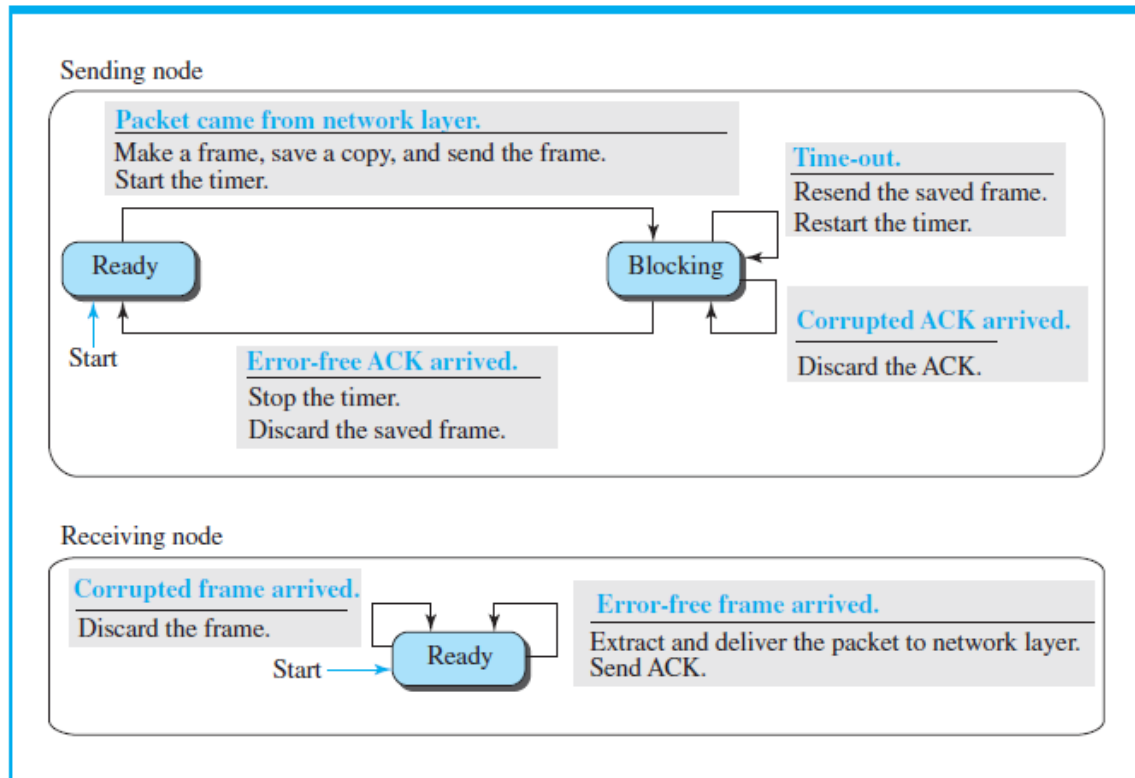


Figure 11.11 shows the FSMs for our primitive Stop-and-Wait protocol.

Figure 11.11 *FSM for the Stop-and-Wait protocol*



Sender States

The sender is initially in the ready state, but it can move between the ready and blocking state.

- ❑ **Ready State.** When the sender is in this state, it is only waiting for a packet from the network layer. If a packet comes from the network layer, the sender creates a frame, saves a copy of the frame, starts the only timer and sends the frame. The sender then moves to the blocking state.
- ❑ **Blocking State.** When the sender is in this state, three events can occur:
 - a. If a time-out occurs, the sender resends the saved copy of the frame and restarts the timer.
 - b. If a corrupted ACK arrives, it is discarded.
 - c. If an error-free ACK arrives, the sender stops the timer and discards the saved copy of the frame. It then moves to the ready state.

Receiver

The receiver is always in the *ready* state. Two events may occur:

- a. If an error-free frame arrives, the message in the frame is delivered to the network layer and an ACK is sent.
- b. If a corrupted frame arrives, the frame is discarded.

Summary

Feature	Stop-and-Wait ARQ
Transmission style	One frame at a time, wait for ACK
Sequence numbers	1-bit (alternates between 0 and 1)
Error control	Timeout and retransmission on no ACK
Efficiency	Low in high-latency or high-bandwidth channels
Complexity	Very simple

Great! Let's dive into the other two important ARQ protocols that improve upon the inefficiencies of Stop-and-Wait:

1. Go-Back-N ARQ Protocol

How Go-Back-N ARQ Works:

- The sender can send **multiple frames** (up to a window size NN) without waiting for an ACK for each one.
 - The sender maintains a **sending window** of size NN , meaning it can send frames $n, n+1, \dots, n+N-1$ consecutively.
 - The receiver only sends cumulative acknowledgments: it sends an ACK for the **last correctly received frame in sequence**.
 - If a frame is lost or corrupted, the receiver discards that frame and **all subsequent frames**, even if they were received correctly.
 - The sender, upon detecting a missing ACK or timeout, **goes back** and retransmits the missing frame and all subsequent frames in the window.
-

Key Points:

- **Sequence numbers:** More bits than Stop-and-Wait to accommodate window size NN .
- **Sender's window size:** NN frames.
- **Receiver's window size:** Usually 1 (only accepts frames in order).
- **ACKs:** Cumulative (ACK number indicates all frames up to that number received correctly).
- **Efficiency:** Much higher than Stop-and-Wait because multiple frames can be "in flight".

Illustration:

Suppose $N=4$:

Frame sent

0 1 2 3 4 5 ...

Sent by sender up to 4 frames without waiting for ACK ✓ ✓ ✓ ✓

If frame 2 is lost, receiver discards frame 3 and 4 even if received. Sender times out and resends frame 2, 3, 4.

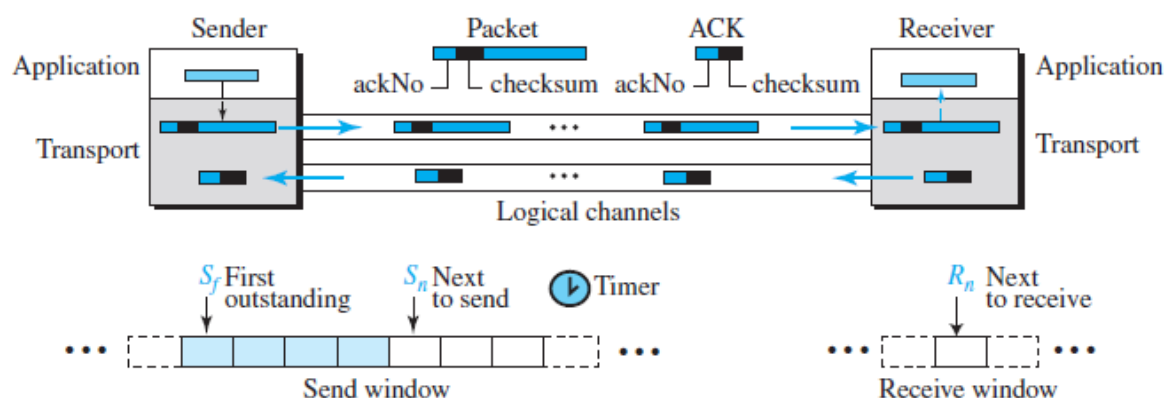
Advantages of Go-Back-N:

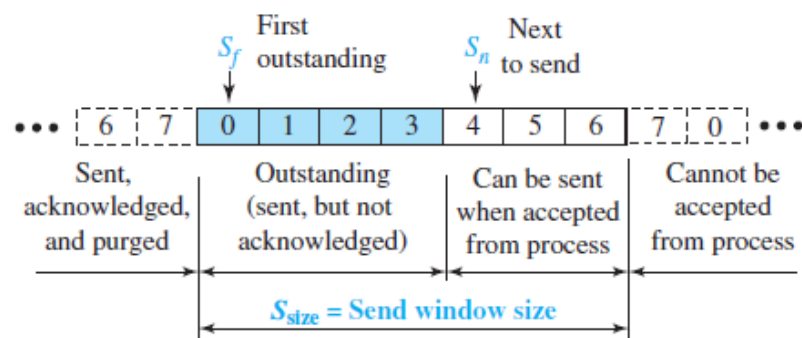
- Improved utilization of the channel (due to pipelining).
 - Simple to implement.
-

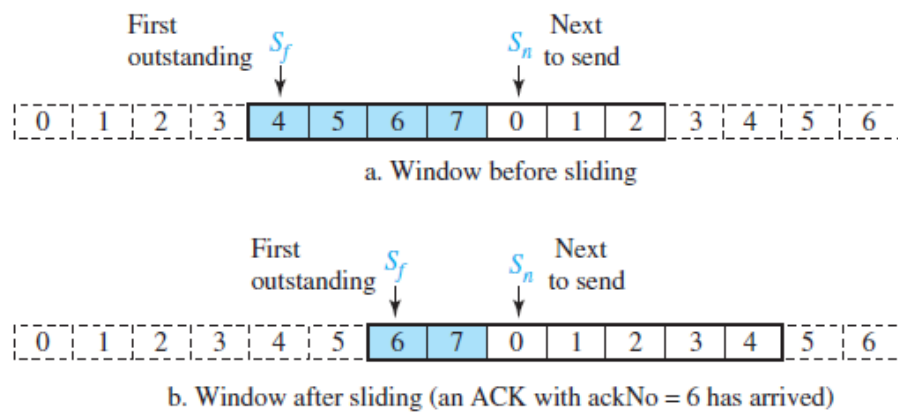
Disadvantages:

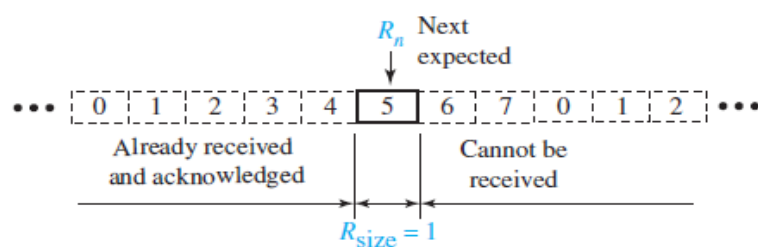
- Retransmits many frames even if only one frame is lost, causing inefficiency.
 - Receiver cannot accept out-of-order frames.
-

Figure 23.23 *Go-Back-N protocol*









The receive window is an abstract concept defining an imaginary box of size 1 with a single variable R_n . The window slides when a correct packet has arrived; sliding occurs one slot at a time.

Resending packets

When the timer expires, the sender resends all outstanding packets. For example, suppose the sender has already sent packet 6 ($S_n = 7$), but the only timer expires. If $S_f = 3$, this means that packets 3, 4, 5, and 6 have not been acknowledged; the sender goes back and resends packets 3, 4, 5, and 6. That is why the protocol is called *Go-Back-N*. On a time-out, the machine goes back N locations and resends all packets.

FSMs

Figure 23.27 shows the FSMs for the GBN protocol.

Sender

The sender starts in the ready state, but thereafter it can be in one of the two states: *ready* or *blocking*. The two variables are normally initialized to 0 ($S_f = S_n = 0$).

- **Ready state.** Four events may occur when the sender is in ready state.
 - a. If a request comes from the application layer, the sender creates a packet with the sequence number set to S_n . A copy of the packet is stored, and the packet is sent. The sender also starts the only timer if it is not running. The value of S_n is now incremented, $(S_n = S_n + 1) \text{ modulo } 2^m$. If the window is full, $S_n = (S_f + S_{\text{size}}) \text{ modulo } 2^m$, the sender goes to the blocking state.
 - b. If an error-free ACK arrives with ackNo related to one of the outstanding packets, the sender slides the window (set $S_f = \text{ackNo}$), and if all outstanding packets are acknowledged ($\text{ackNo} = S_n$), then the timer is stopped. If all outstanding packets are not acknowledged, the timer is restarted.

2. Selective Repeat ARQ Protocol

How Selective Repeat Works:

- Similar to Go-Back-N, the sender can send multiple frames within a window.
 - The receiver **accepts frames out of order** and buffers them.
 - Only **frames detected as lost or corrupted are retransmitted**.
 - ACKs are sent for each correctly received frame (individual ACKs).
 - Sender keeps track of which frames are acknowledged and retransmits **only those that are lost**.
-

Key Points:

- **Sequence numbers:** Larger range (at least twice the window size) to avoid ambiguity.
 - **Sender's and receiver's window size:** Both are $2N$.
 - **ACKs:** Individual for each frame.
 - **Buffers:** Receiver needs buffer to store out-of-order frames.
-

Figure 23.31 *Outline of Selective-Repeat*

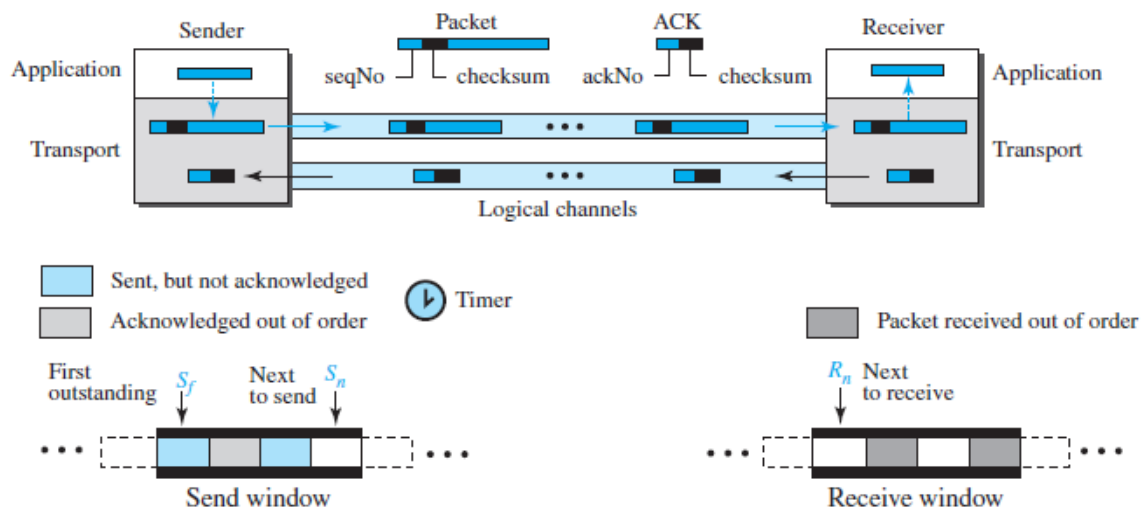
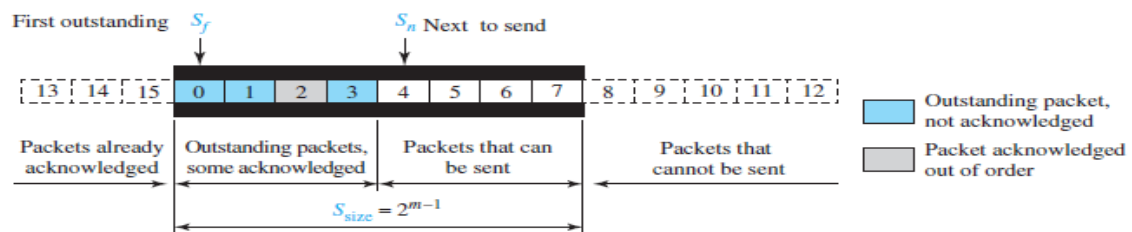


Figure 23.32 *Send window for Selective-Repeat protocol*



window that are shaded define packets that have arrived out of order and are waiting for the earlier transmitted packet to arrive before delivery to the application layer.

Figure 23.33 *Receive window for Selective-Repeat protocol*

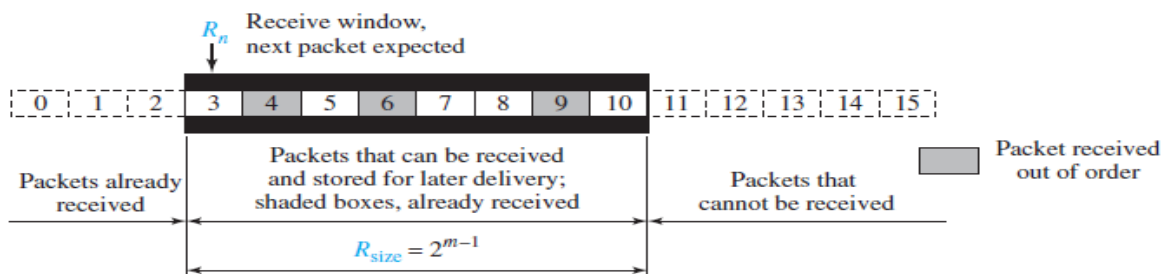


Illustration:

Frame sent 0 1 2 3 4 5 ...

Frames sent within window size NN ✓ ✓ ✓ ✓

If frame 2 is lost, receiver buffers frame 3 and 4, sends ACKs for 0,1,3,4. Sender retransmits frame 2 only.

Advantages of Selective Repeat:

- Much better efficiency than Go-Back-N.
 - Only lost frames are retransmitted.
 - Receiver can process frames out of order.
-

Disadvantages:

- More complex to implement.
 - Requires buffer space on receiver side.
-

Summary Comparison Table

Feature	Stop-and-Wait	Go-Back-N	Selective Repeat
Window size	1	$N > 1$	$N > 1$
ACK type	Individual	Cumulative	Individual
Receiver buffer	1 frame (in order only)	1 frame (in order only)	Multiple frames (out-of-order)
Retransmission policy	One frame at a time	Retransmit from error frame onward	Retransmit only errored frames
Efficiency	Low	Medium	High
Complexity	Low	Medium	High

Awesome! Let's walk through some clear examples of each ARQ protocol: **Stop-and-Wait**, **Go-Back-N**, and **Selective Repeat** — so you can see how they operate in practice.

1. Stop-and-Wait ARQ Example

Scenario:

- Sender wants to send frames: 0, 1, 0, 1 (sequence numbers alternate).
 - Frame 1 gets lost.
 - Assume timeout is triggered after frame 1 loss.
-

Step	Sender Action	Receiver Action	Notes
1	Send Frame 0	Receive Frame 0	Frame 0 received correctly
2	Wait for ACK 0	Send ACK 0	Sender receives ACK 0, sends next frame
3	Send Frame 1	Frame 1 lost in transmission	No ACK sent back
4	Timeout expires -		Sender retransmits Frame 1
5	Send Frame 1	Receive Frame 1	Frame 1 received correctly
6	Wait for ACK 1	Send ACK 1	Sender receives ACK 1, sends next frame
7	Send Frame 0	Receive Frame 0	Frame 0 received correctly

Key points:

- Sender waits for ACK before sending next frame.
 - Lost frame triggers timeout and retransmission.
 - Inefficient, especially when frames are lost.
-

2. Go-Back-N ARQ Example

Scenario:

- Window size $N=4$.
- Frames 0, 1, 2, 3 sent in succession.
- Frame 2 gets lost.
- Receiver discards frames 3 and 4.

Step	Sender Action	Receiver Action	Notes
1	Send Frames 0-3	Receive Frames 0,1 ACK 1 sent (all up to frame 1 received)	
2	Continue sending Frame 4	Frame 2 lost	Frame 3 discarded (out of order)
3	Timeout for Frame 2	-	Sender retransmits Frame 2
4	Retransmit Frame 2	Receive Frame 2	ACK 3 sent (now frames 0,1,2,3 received)
5	Continue with Frame 5	Receive Frame 4	ACK 4 sent

Key points:

- Multiple frames are sent without waiting for ACK.
- Lost frame causes retransmission of that frame and all subsequent frames.
- Receiver only accepts frames in order.

3. Selective Repeat ARQ Example

Scenario:

- Window size $N=4$
- Frames 0, 1, 2, 3 sent.
- Frame 2 lost.
- Receiver buffers out-of-order frames 3 and 4.

Step	Sender Action	Receiver Action	Notes
1	Send Frames 0-3	Receive Frames 0,1	Send ACK 0, ACK 1
2	Continue sending Frame 4	Frame 2 lost	Receive Frame 3, buffer it, send ACK 3
3	Retransmit Frame 2	Receive Frame 2	Deliver frames 2, 3 in order
4	Send next frames	Send ACKs for all correctly received frames	

Key points:

- Receiver buffers out-of-order frames.
 - Only lost frames are retransmitted.
 - Much better bandwidth utilization.
-

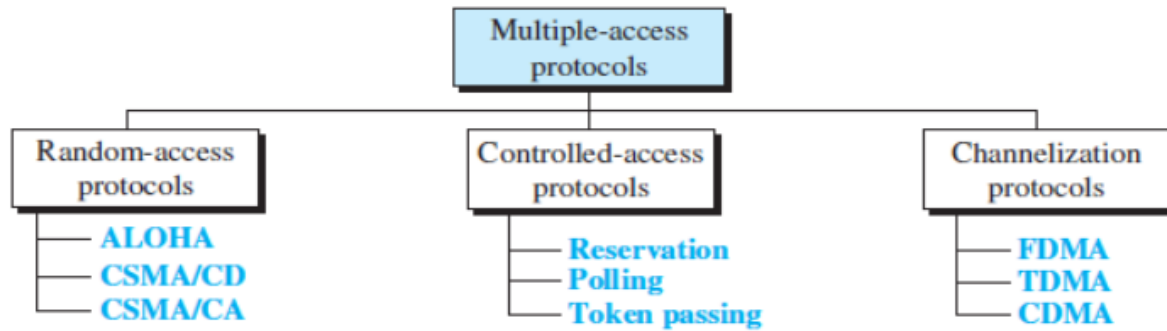
Visual Summary

Protocol	Action on lost Frame	Frame acceptance	Retransmission
Stop-and-Wait	Retransmit one frame only	One at a time, in order	After timeout for each frame
Go-Back-N	Retransmit lost frame + all after	In order only	After timeout for lost frame
Selective Repeat	Retransmit lost frames only	Out-of-order frames buffered	Individual retransmission

MEDIA ACCESS CONTROL (MAC) PROTOCOLS

The data-link layer is divided into two sub layers: data-link control (DLC) and media access control. When we are using a dedicated link, such as a dial-up telephone line, we need only a data-link-control protocol, such as the Point-to-Point Protocol (PPP), that manages the data transfer between the two ends. On the other hand, if we are sharing the media, wire or air, with other users, we need to have a protocol to first manage the sharing process and then to do the data transfer. For example, if we use our cellular phone to connect to another cellular phone, the channel (the band allocated to the vendor company) is not dedicated. A person a few feet away from us may be using the same band to talk to her friend.

When nodes or stations are connected and use a common link, called a multipoint or broadcast link, we need a multiple-access protocol to coordinate access to the link. Many protocols have been devised to handle access to a shared link. We categorize them into three groups.



Taxonomy of Multiple Access Protocols

1. Random Access

In **random-access** or **contention methods**, no station is superior to another station and none is assigned control over another. At each instance, a station that has data to send uses a procedure defined by the protocol to make a decision on whether or not to send. This decision depends on the state of the medium (idle or busy). There is no scheduled time for a station to transmit. Transmission is random among the stations. Also no rules specify which station should send next. Stations compete with one another to access the medium. Hence they are called **Random Access protocols**.

In a random-access method, each station has the right to the medium without being controlled by any other station. However, if more than one station tries to send, there is an access conflict—*collision*—and the frames will be either destroyed or modified. To avoid access conflict or to resolve it when it happens, each station follows a procedure that answers the following questions:

When can the station access the medium?

What can the station do if the medium is busy?

How can the station determine the success or failure of the transmission?

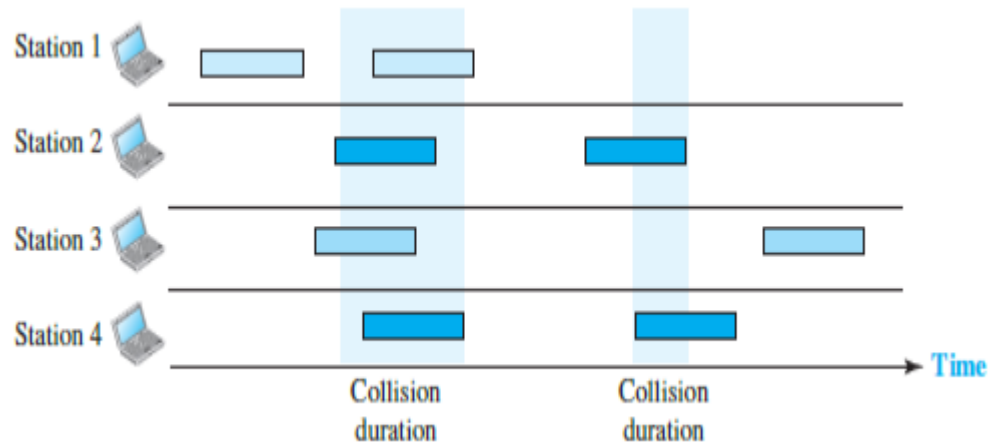
What can the station do if there is an access conflict?

1.1 ALOHA

ALOHA, the earliest random-access method, was developed at the University of Hawaii in early 1970. It was designed for a radio (wireless) LAN, but it can be used on any shared medium. It is obvious that there are potential collisions in this arrangement. The medium is shared between the stations. When a station sends data, another station may attempt to do so at the same time. The data from the two stations collide and become garbled.

Pure ALOHA

The original ALOHA protocol is called **pure ALOHA**. This is a simple but elegant protocol. The idea is that each station sends a frame whenever it has a frame to send (multiple access).



Frames in Pure ALOHA Network

There are four stations (unrealistic assumption) that contend with one another for access to the shared channel. Above figure shows that each station sends two frames; There are a total of eight frames on the shared medium.

Some of these frames collide because multiple frames are in contention for the shared channel. Only two frames survive:

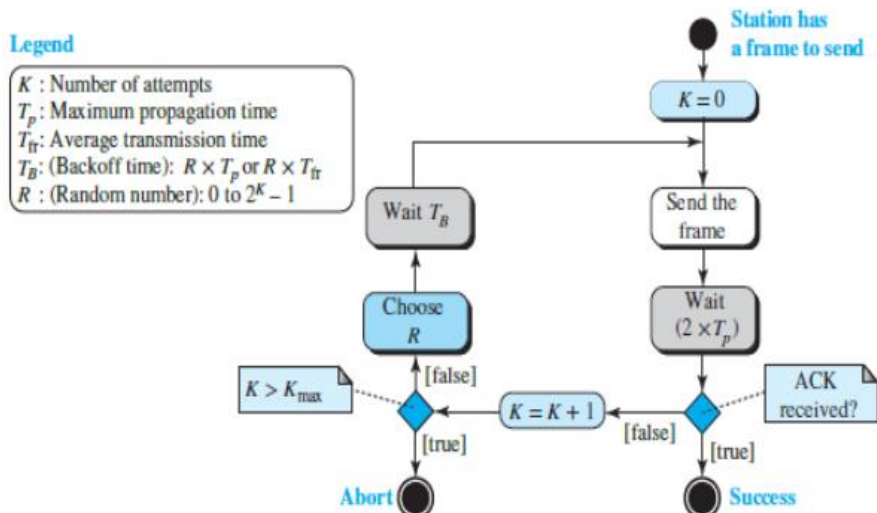
one frame from station 1 and one frame from station 3. We need to mention that even if 1 bit of a frame coexists on the channel with 1 bit from another frame, there is a collision and both will be destroyed. It is obvious that we need to resend the frames that have been destroyed during transmission.

The pure ALOHA protocol relies on acknowledgments from the receiver. When a station sends a frame, it expects the receiver to send an acknowledgment. If the acknowledgment does not arrive after a time-out period, the station assumes that the frame (or the acknowledgment) has been destroyed and resends the frame.

A collision involves two or more stations. If all these stations try to resend their frames after the time-out, the frames will collide again. Pure ALOHA dictates that when the timeout period passes, each station waits a random amount of time before resending its frame.

The randomness will help avoid more collisions. We call this time **the backoff time T_B** .

Pure ALOHA has a second method to prevent congesting the channel with retransmitted frames. After a maximum number of retransmission attempts K_{max} , a station must give up and try later. Below figure shows the procedure for pure ALOHA.



Procedure for Pure ALOHA

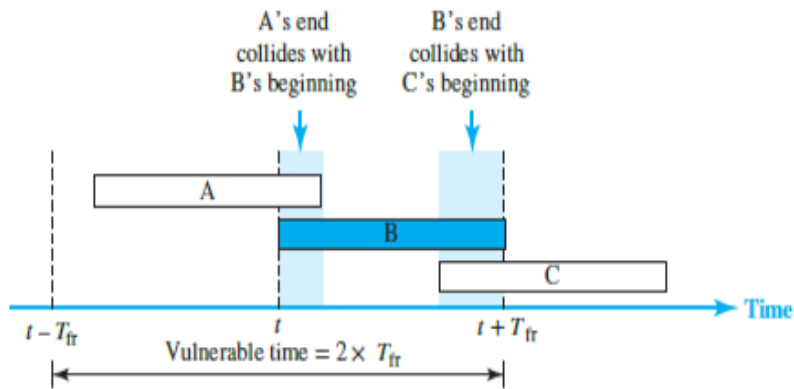
The time-out period is equal to the maximum possible round-trip propagation delay, which is twice the amount of time required to send a frame between the two most widely separated stations ($2 \times T_p$). **The backoff time T_B is a random value that normally depends on K (the number of attempted unsuccessful transmissions).** The formula for T_B depends on the implementation.

One common formula is the *binary exponential backoff*. In this method, for each retransmission, a multiplier $R = 0$ to $2K - 1$ is randomly chosen and multiplied by T_p (maximum propagation time) or T_{fr} (the average time required to send out a frame) to find T_B . Note that in this procedure, the range of the random numbers increases after each collision. The value of K_{max} is usually chosen as 15.

Example

The stations on a wireless ALOHA network are a maximum of 600 km apart. If we assume that signals propagate at 3×10^8 m/s, we find $T_p = (600 \times 10^3) / (3 \times 10^8) = 2$ ms. For $K = 2$, the range of R is $\{0, 1, 2, 3\}$. This means that T_B can be 0, 2, 4, or 6 ms, based on the outcome of the random variable R .

Vulnerable time Let us find the length of time, the *vulnerable time*, in which there is a possibility of collision. We assume that the stations send fixed-length frames with each frame taking T_{fr} seconds to send. Below figure shows the vulnerable time for stationB.



Vulnerable time for Pure ALOHA Protocol

Station B starts to send a frame at time t . Now imagine station A has started to send its frame after $t - T_{fr}$. This leads to a collision between the frames from station B and station A.

On the other hand, suppose that station C starts to send a frame before time $t + T_{fr}$. Here, there is also a collision between frames from station B and station C. Looking at the above figure, we see that the vulnerable time, during which a collision may occur in pure ALOHA, is 2 times the frame transmission time.

Pure ALOHA vulnerable time = $2 \times T_{fr}$

Example:

A pure ALOHA network transmits 200-bit frames on a shared channel of 200 kbps. What is the requirement to make this frame collision-free?

Solution: The average frame transmission time T_{fr} is 200 bits/200 kbps or 1 ms. The vulnerable time is $2 \times 1 \text{ ms} = 2 \text{ ms}$. This means no station should send later than 1 ms before this station starts transmission and no station should start sending during the period (1 ms) that this station is sending.

Throughput

Let us call G the average number of frames generated by the system during one frame transmission time. Then it can be proven that the average number of successfully transmitted frames for **pure ALOHA** is $S = G \times e^{-2G}$.

The maximum throughput S_{max} is 0.184, for $G = 1/2$. (We can find it by setting the derivative of S with respect to G to 0.) In other words, if one-half a frame is generated during one frame transmission time (one frame during two frame transmission times), then 18.4 percent of these frames reach their destination successfully. We expect $G = 1/2$ to produce the maximum throughput because the vulnerable time is 2 times the frame transmission time. Therefore, if a station generates only one frame in this vulnerable time (and no other stations generate a frame during this time), the frame will reach its destination successfully.

The throughput for pure ALOHA is $S = G \times e^{-2G}$.
 The maximum throughput $S_{max} = 1/(2e) = 0.184$ when $G = 1/2$.

Example

A pure ALOHA network transmits 200-bit frames on a shared channel of 200 kbps. Find the throughput for each of the following frames per second produced by the system.

- a. 1000
- b. 500
- c. 250

Solution

The frame transmission time is $200/200$ kbps or 1 ms.

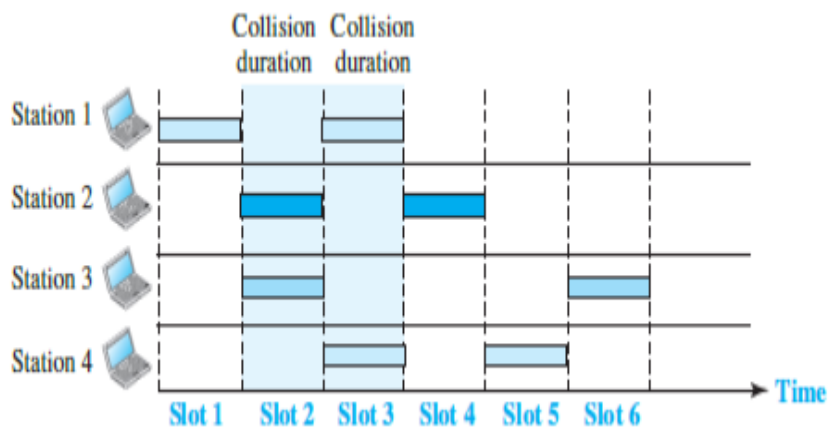
a. If the system creates 1000 frames per second, or 1 frame per millisecond, then $G = 1$. In this case, $S = G \times e^{-2G} = 0.135$ (13.5 percent). This means that the throughput is $1000 \times 0.135 = 135$ frames. Only 135 frames out of 1000 will probably survive.

b. If the system creates 500 frames per second, or $1/2$ frame per millisecond, then $G = 1/2$. In this case, $S = G \times e^{-2G} = 0.184$ (18.4 percent). This means that the throughput is $500 \times 0.184 = 92$ and that only 92 frames out of 500 will probably survive. Note that this is the *maximum* throughput case, percentage-wise.

c. If the system creates 250 frames per second, or $1/4$ frame per millisecond, then $G = 1/4$. In this case, $S = G \times e^{-2G} = 0.152$ (15.2 percent). This means that the throughput is $250 \times 0.152 = 38$. Only 38 frames out of 250 will probably survive.

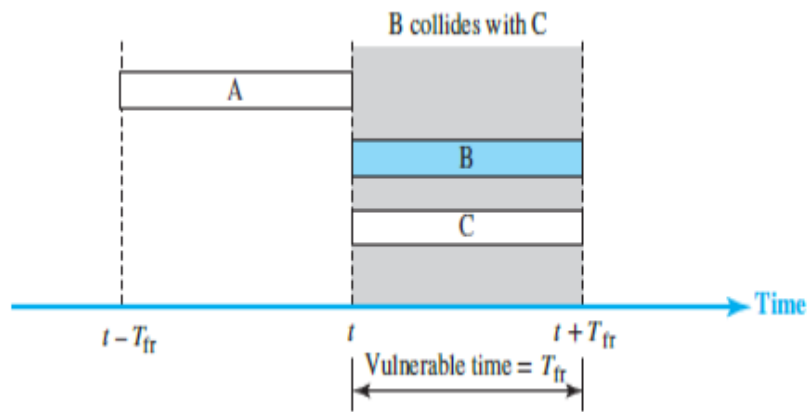
Slotted ALOHA

In **slotted ALOHA** we divide the time into slots of T_{fr} seconds and force the station to send only at the beginning of the time slot. Below figure shows an example of frame collisions in slotted ALOHA.



Frames in a Slotted ALOHA Network

Because a station is allowed to send only at the beginning of the synchronized time slot, if a station misses this moment, it must wait until the beginning of the next time slot. This means that the station that started at the beginning of this slot has already finished sending its frame. Of course, there is still the possibility of collision if two stations try to send at the beginning of the same time slot. However, the vulnerable time is now reduced to one-half, equal to T_{fr} . Below figure shows this situation.



Vulnerable Time for Slotted ALOHA Protocol

Throughput It can be proven that the average number of successful transmissions for slotted ALOHA is $S = G \times e^{-G}$. The maximum throughput S_{max} is 0.368, when $G = 1$. In other words, if one frame is generated during one frame transmission time, then 36.8 percent of these frames reach their destination successfully. We expect $G = 1$ to produce maximum throughput because the vulnerable time is equal to the frame transmission time.

Slotted ALOHA vulnerable time = T_{fr}

Therefore, if a station generates a frame during this time (and no other station generates a frame during this time), the frame will reach its destination Successfully.

**The throughput for slotted ALOHA is $S = G \times e^{-G}$.
The maximum throughput $S_{max} = 0.368$ when $G = 1$.**

Example:

A slotted ALOHA network transmits 200-bit frames using a shared channel with a 200-kbps bandwidth. Find the throughput for each of the following frames per second produced by the system (all stations together).

- a. 1000
- b. 500
- c. 250

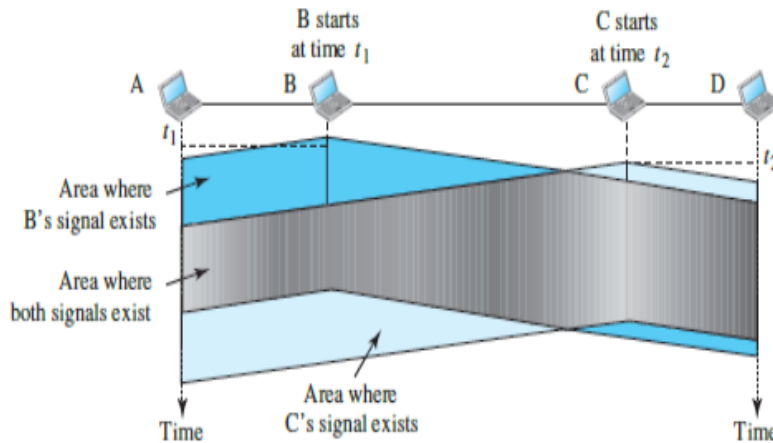
Solution

This situation is similar to Exercise 3.10 except that the network is using slotted ALOHA instead of pure ALOHA. The frame transmission time is $200/200$ kbps or 1 ms.

- a. In this case G is 1. So $S = G \times e^{-G} = 0.368$ (36.8 percent). This means that the throughput is $1000 \times 0.0368 = 368$ frames. Only 368 out of 1000 frames will probably survive. Note that this is the maximum throughput case, percentage-wise.
- b. Here G is $1/2$. In this case $S = G \times e^{-G} = 0.303$ (30.3 percent). This means that the throughput is $500 \times 0.0303 = 151$. Only 151 frames out of 500 will probably survive.
- c. Now G is $1/4$. In this case $S = G \times e^{-G} = 0.195$ (19.5 percent). This means that the throughput is $250 \times 0.195 = 49$. Only 49 frames out of 250 will probably survive.

1.1 Carrier Sense Multiple Access (CSMA)

In CSMA, the chance of collision can be reduced if a station senses the medium before trying to use it. **Carrier sense multiple access (CSMA)** requires that each station first listen to the medium (or check the state of the medium) before sending. In other words, CSMA is based on the principle “sense before transmit” or “listen before talk.”



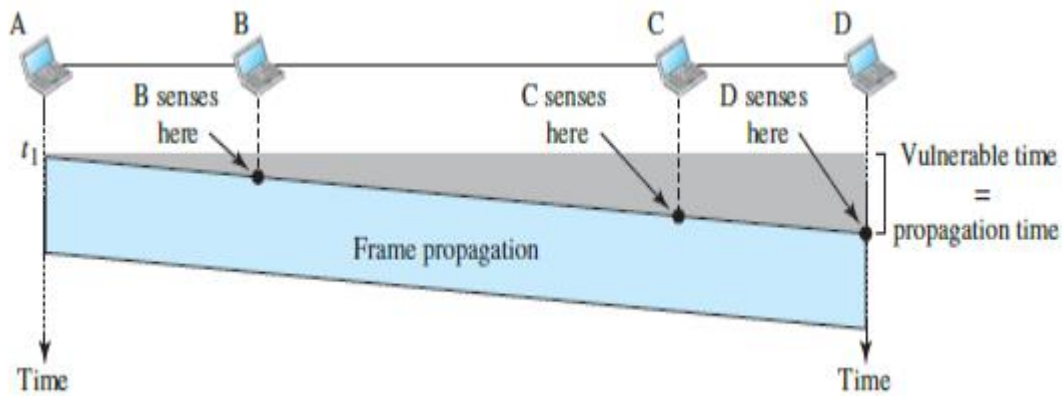
Space and Time Model of Collision in CSMA

CSMA can reduce the possibility of collision, but it cannot eliminate it. The above figure shows a space and time model of a CSMA network. Stations are connected to a shared channel (usually a dedicated medium). The possibility of collision still exists because of propagation delay; when a station sends a frame, it still takes time (although very short) for the first bit to reach every station and for every station to sense it. At time t_1 , station B senses the medium and finds it idle, so it sends a frame. At time t_2 ($t_2 > t_1$), station C senses the medium and finds it idle because, at this time, the first bits from station B have not reached station C. Station C also sends a frame. The two signals collide, and both frames are destroyed.

Vulnerable Time

The vulnerable time for CSMA is the *propagation time* T_p . This is the time needed for a signal to propagate from one end of the medium to the other. When a station sends a frame and any other station tries to send a frame during this time, a collision will result. But if the first bit of the frame reaches the end of the medium, every station will already have heard the bit and will refrain from sending.

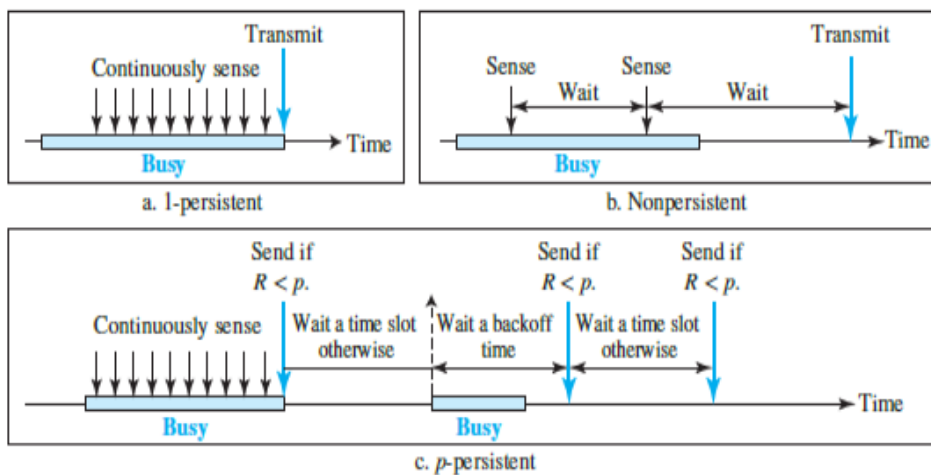
Below figure shows the worst case. The leftmost station A sends a frame at time t_1 , which reaches the rightmost station D at time $t_1 + T_p$. The gray area shows the vulnerable area in time and space.



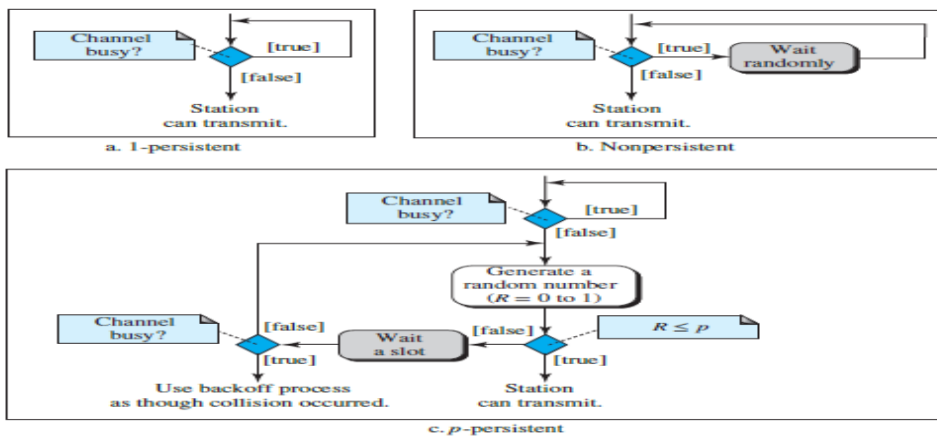
Vulnerable Time in CSMA

Persistence Methods

What should a station do if the channel is busy? What should a station do if the channel is idle? Three methods have been devised to answer these questions: **1-persistent method**, **non persistent method**, and **p-persistent method**. Below figure shows the behaviour of these three persistence methods when a station finds a channel busy.



Behaviour of three Persistence Methods



Flow diagram for three persistence methods

1-Persistent The *1-persistent method* is simple and straightforward. In this method, after the station finds the line idle, it sends its frame immediately (with probability 1). This method has the highest chance of collision because two or more stations may find the line idle and send their frames immediately. We will see later that Ethernet uses this method.

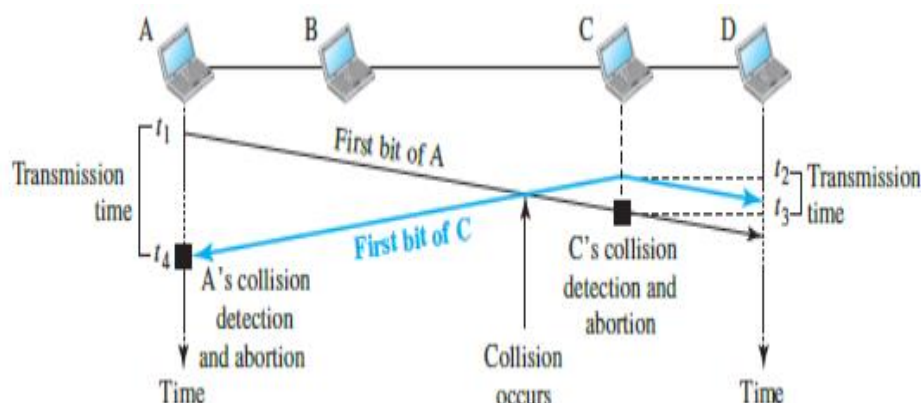
Nonpersistent In the *nonpersistent method*, a station that has a frame to send senses the line. If the line is idle, it sends immediately. If the line is not idle, it waits a random amount of time and then senses the line again. The nonpersistent approach reduces the chance of collision because it is unlikely that two or more stations will wait the same amount of time and retry to send simultaneously. However, this method reduces the efficiency of the network because the medium remains idle when there may be stations with frames to send.

p-Persistent The *p-persistent method* is used if the channel has time slots with a slot duration equal to or greater than the maximum propagation time. The *p-persistent* approach combines the advantages of the other two strategies. It reduces the chance of collision and improves efficiency. In this method, after the station finds the line idle, it follows these steps:

1. With probability p , the station sends its frame.
2. With probability $q = 1 - p$, the station waits for the beginning of the next time slot and checks the line again.
 - a. If the line is idle, it goes to step 1.
 - b. If the line is busy, it acts as though a collision has occurred and uses the backoff procedure.

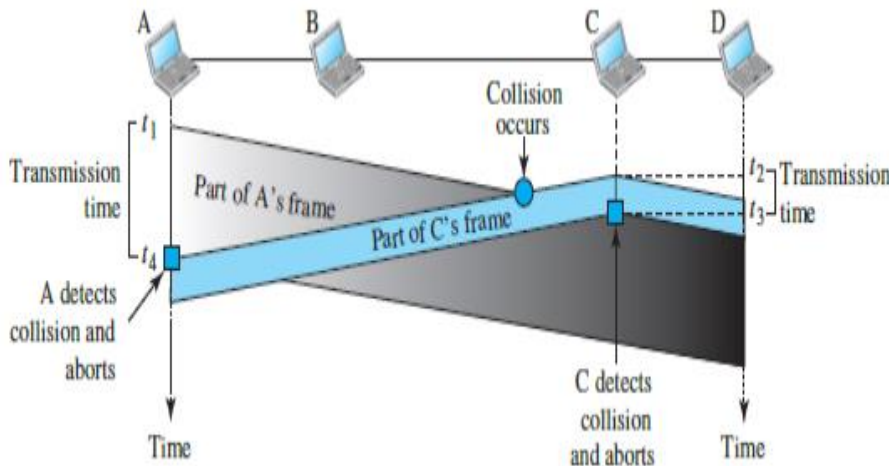
1.2 CSMA/CD

The CSMA method does not specify the procedure following a collision. **Carrier sense multiple access with collision detection (CSMA/CD)** augments the algorithm to handle the collision. In this method, a station monitors the medium after it sends a frame to see if the transmission was successful. If so, the station is finished. If, however, there is a collision, the frame is sent again. Although each station continues to send bits in the frame until it detects the collision, we show what happens as the first bits collide. In below figure, stations A and C are involved in the collision:



Collision of the first bits in CSMA/CD

At time t_1 , station A has executed its persistence procedure and starts sending the bits of its frame. At time t_2 , station C has not yet sensed the first bit sent by A. Station C executes its persistence procedure and starts sending the bits in its frame, which propagate both to the left and to the right. The collision occurs sometime after time t_2 . Station C detects a collision at time t_3 when it receives the first bit of A's frame. Station C immediately (or after a short time, but we assume immediately) aborts transmission. Station A detects a collision at time t_4 when it receives the first bit of C's frame; it also immediately aborts transmission. A transmits for the duration $t_4 - t_1$; C transmits for the duration $t_3 - t_2$.



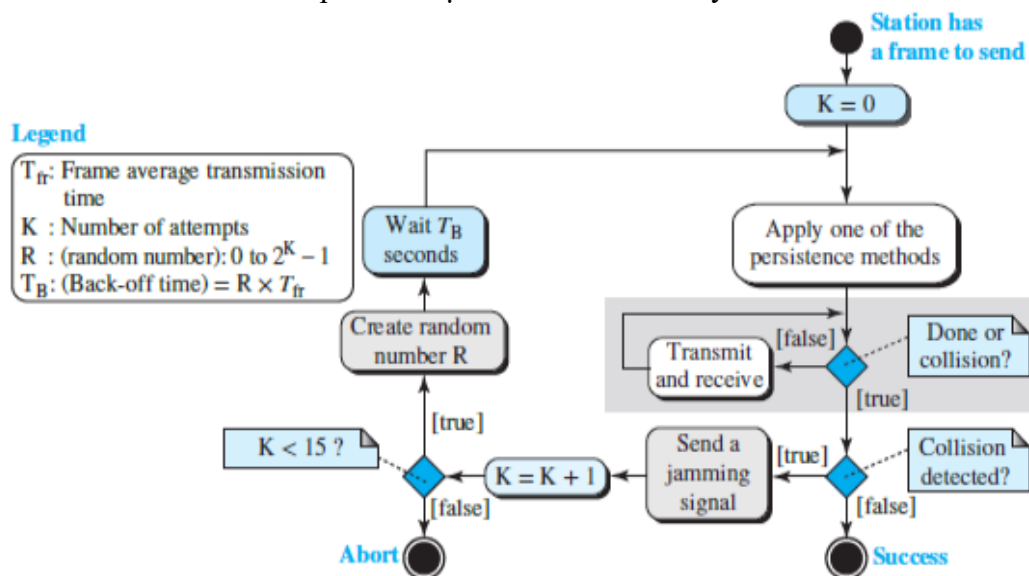
Collision and abortion in CSMA/CD

Example

A network using CSMA/CD has a bandwidth of 10 Mbps. If the maximum propagation time (including the delays in the devices and ignoring the time needed to send a jamming signal, as we see later) is $25.6 \mu\text{s}$, what is the minimum size of the frame?

Solution

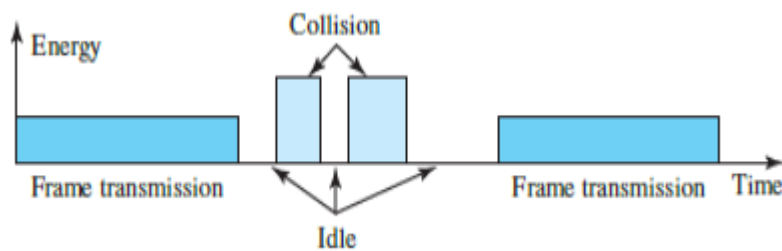
The minimum frame transmission time is $T_{fr} = 2 \times T_p = 51.2 \mu\text{s}$. This means, in the worst case, a station needs to transmit for a period of $51.2 \mu\text{s}$ to detect the collision. The minimum size of the frame is $10 \text{ Mbps} \times 51.2 \mu\text{s} = 512 \text{ bits}$ or 64 bytes.



Flow diagram for the CSMA/CD

Energy Level

We can say that the level of energy in a channel can have three values: zero, normal, and abnormal. At the zero level, the channel is idle. At the normal level, a station has successfully captured the channel and is sending its frame. At the abnormal level, there is a collision and the level of the energy is twice the normal level. A station that has a frame to send or is sending a frame needs to monitor the energy level to determine if the channel is idle, busy, or in collision mode. Below figure shows the situation:



Energy level during transmission, idleness, or collision

Throughput

The throughput of CSMA/CD is greater than that of pure or slotted ALOHA. The maximum throughput occurs at a different value of G and is based on the persistence method and the value of p in the p -persistent approach. For the 1-persistent method, the maximum throughput is around 50 percent when $G = 1$. For the non persistent method, the maximum throughput can go up to 90 percent when G is between 3 and 8.

1.3 CSMA/CA

A variation of the CSMA method is **Carrier Sense Multiple Access with Collision Avoidance (CSMA/CA)**,

1. Controlled Access

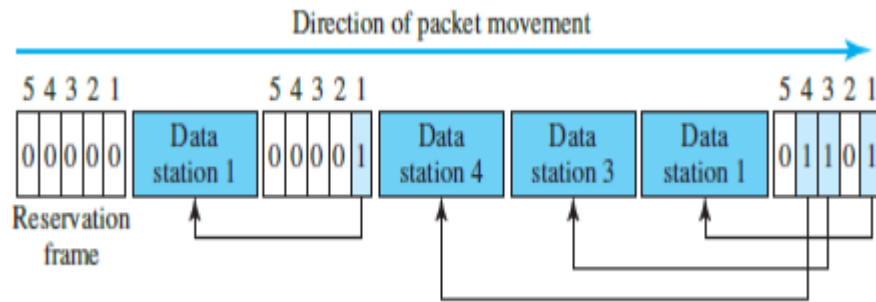
In **controlled access**, the stations consult one another to find which station has the right to send. A station cannot send unless it has been authorized by other stations.

Reservation

In the **reservation** method, a station needs to make a reservation before sending data. Time is divided into intervals. In each interval, a reservation frame precedes the data frames sent in that interval.

If there are N stations in the system, there are exactly N reservation minislots in the reservation frame. Each minislot belongs to a station. When a station needs to send a data frame, it makes a reservation in its own minislot. The stations that have made reservations can send their data frames after the reservation frame.

Below figure shows a situation with five stations and a five-minislot reservation frame. In the first interval, only stations 1, 3, and 4 have made reservations. In the second interval, only station 1 has made a reservation



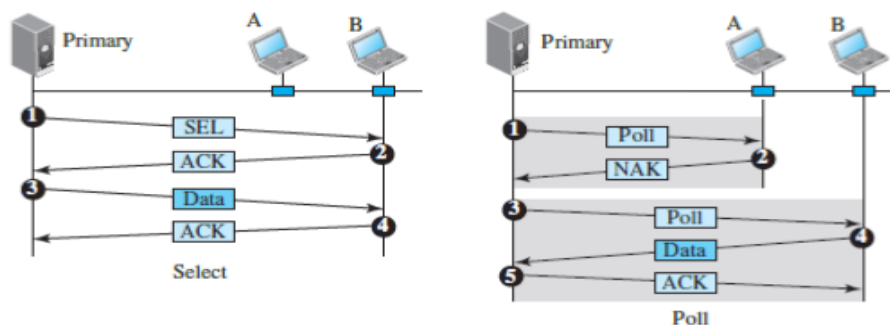
Reservation access method

Polling

Polling works with topologies in which one device is designated as a *primary station* and the other devices are *secondary stations*. All data exchanges must be made through the primary device even when the ultimate destination is a secondary device.

The primary device controls the link; the secondary devices follow its instructions. It is up to the primary device to determine which device is allowed to use the channel at a given time.

The primary device, therefore, is always the initiator of a session. This method uses poll and select functions to prevent collisions. However, the drawback is if the primary station fails, the system goes down.



Select and poll functions in the polling-access method

Select

The *select* function is used whenever the primary device has something to send. If the primary is neither sending nor receiving data, it knows the link is available. If it has something to send, the primary device sends it. However, it does not know that the target device is prepared to receive. So the primary must alert the secondary to the upcoming transmission and wait for an acknowledgment of the secondary's ready status. Before sending data, the primary creates and transmits a select (SEL) frame, one field of which includes the address of the intended secondary.

Poll

The *poll* function is used by the primary device to solicit transmissions from the secondary devices. When the primary is ready to receive data, it must ask (poll) each device in turn if it has anything to send. When the first secondary is approached, it responds either with a NAK frame if it has nothing to send or with data (in the form of a data frame) if it does. If the response is negative (a NAK frame), then the primary polls the next secondary in the same

manner until it finds one with data to send. When the response is positive (a data frame), the primary reads the frame and returns an acknowledgment (ACK frame) verifying its receipt.

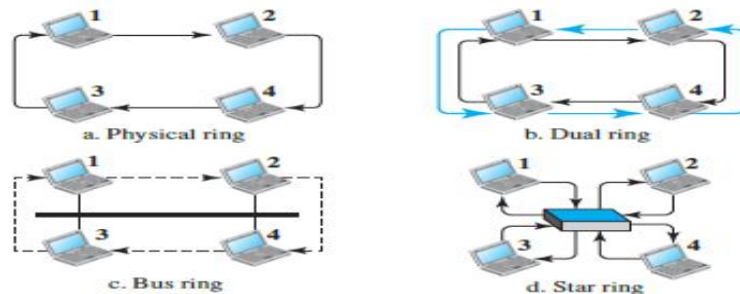
Token Passing

In the **token-passing** method, the stations in a network are organized in a logical ring. That is, for each station, there is a *predecessor* and a *successor*. The predecessor is the station that is logically before the station in the ring; the successor is the station that is after the station in the ring. The current station is the one that is accessing the channel now. The right to this access has been passed from the predecessor to the current station. The right will be passed to the successor when the current station has no more data to send.

In this method, a special packet called a *token* circulates through the ring. The possession of the token gives the station the right to access the channel and send its data. When a station has some data to send, it waits until it receives the token from its predecessor. It then holds the token and sends its data. When the station has no more data to send, it releases the token, passing it to the next logical station in the ring. The station cannot send data until it receives the token again in the next round. In this process, when a station receives the token and has no data to send, it just passes the data to the next station.

Logical Ring

In a token-passing network, stations do not have to be physically connected in a ring; the ring can be a logical one. Below figure shows four different physical topologies that can create a logical ring.



Logical ring and physical topology in token-passing access method

In the physical ring topology, when a station sends the token to its successor, the token cannot be seen by other stations; the successor is the next one in line. This means that the token does not need to have the address of the next successor.

The dual-ring topology uses a second (auxiliary) ring that operates in the reverse direction compared with the main ring. The second ring is for emergencies only (such as a spare tire for a car). If one of the links in the main ring fails, the system automatically combines the two rings to form a temporary ring. After the failed link is restored, the auxiliary ring becomes idle again.

In the bus ring topology, also called a token bus, the stations are connected to a single cable called a bus. They, however, make a logical ring, because each station knows the address of its successor. When a station has finished sending its data, it releases the token and inserts the address of its successor in the token. Only the station with the address matching the destination address of the token gets the token to access the shared media.

In a star ring topology, the physical topology is a star. There is a hub, however, that acts as the connector. The wiring inside the hub makes the ring; the stations are connected to this ring through the two wire connections. This topology makes the network less prone to failure because if a link goes down, it will be bypassed by the hub and the rest of the stations can operate.

Wireless LAN- IEEE 802.11

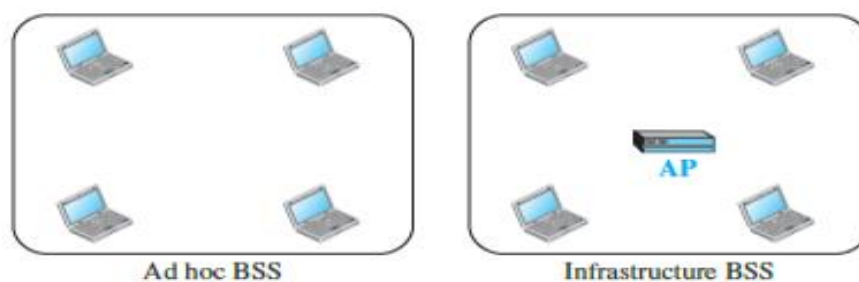
IEEE has defined the specifications for a wireless LAN, called IEEE 802.11, which covers the physical and data-link layers. It is sometimes called *wireless Ethernet*.

Architecture

The IEEE standard defines two kinds of services: the basic service set (BSS) and the extended service set (ESS).

Basic Service Set

IEEE 802.11 defines the **basic service set (BSS)** as the building blocks of a wireless LAN. A basic service set is made up of stationary or mobile wireless stations and an optional central base station, known as the access point (AP).



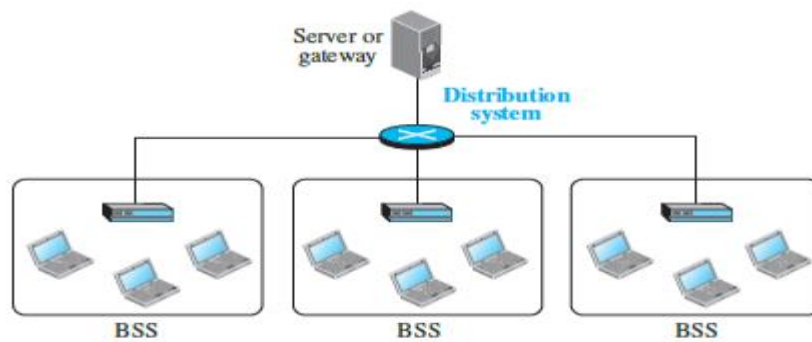
Basic service sets (BSSs)

The BSS without an AP is a stand-alone network and cannot send data to other BSSs. It is called an *ad hoc architecture*. In this architecture, stations can form a network without the need of an AP; they can locate one another and agree to be part of a BSS. A BSS with an AP is sometimes referred to as an *infrastructure BSS*.

Extended Service Set

An **extended service set (ESS)** is made up of two or more BSSs with APs. In this case, the BSSs are connected through a *distribution system*, which is a wired or a wireless network. The distribution system connects the APs in the BSSs. IEEE 802.11 does not restrict the distribution system; it can be any IEEE LAN such as an Ethernet.

When BSSs are connected, the stations within reach of one another can communicate without the use of an AP. However, communication between a station in a BSS and the outside BSS occurs via the AP.



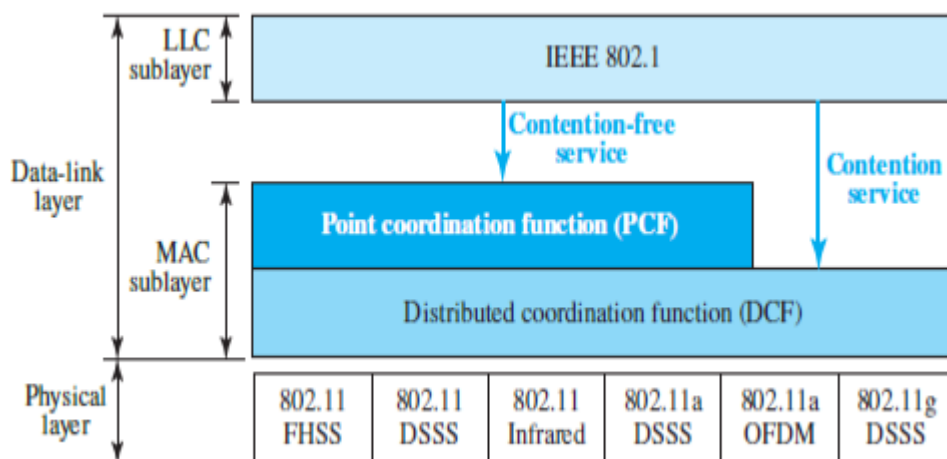
Extended service set (ESS)

Station Types

IEEE 802.11 defines three types of stations based on their mobility in a wireless LAN: **No-transition**, **BSS-transition**, and **ESS-transition mobility**. A station with no-transition mobility is either stationary (not moving) or moving only inside a BSS. A station with BSS transition mobility can move from one BSS to another, but the movement is confined inside one ESS. A station with ESS-transition mobility can move from one ESS to another.

MAC Sublayer

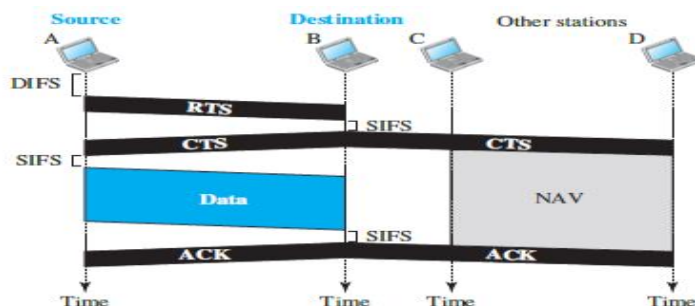
IEEE 802.11 defines two MAC sublayers: the **distributed coordination function (DCF)** and the **point coordination function (PCF)**. Below figure shows the relationship among the two MAC sublayers, the LLC sublayer, and the physical layer.



MAC layers in the IEEE 802.11 standard

Distributed Coordination Function

DCF uses CSMA/CA as the access method. Below figure shows the exchange of data and control frames in time.



CSMA/CA and NAV

1. Before sending a frame, the source station senses the medium by checking the energy level at the carrier frequency.
 - a. The channel uses a persistence strategy with backoff until the channel is idle.
 - b. After the station is found to be idle, the station waits for a period of time called the **DCF interframe space (DIFS)**; then the station sends a control frame called the *request to send (RTS)*.
2. After receiving the RTS and waiting a period of time called the **short interframe space (SIFS)**, the destination station sends a control frame, called the *clear to send (CTS)*, to the source station. This control frame indicates that the destination station is ready to receive data.
3. The source station sends data after waiting an amount of time equal to the SIFS.
4. The destination station, after waiting an amount of time equal to the SIFS, sends an acknowledgment to show that the frame has been received. Acknowledgment is needed in this protocol because the station does not have any means to check for the successful arrival of its data at the destination. On the other hand, the lack of collision in CSMA/CD is a kind of indication to the source that data have arrived.

Network Allocation Vector

When a station sends an RTS frame, it includes the duration of time that it needs to occupy the channel. The stations that are affected by this transmission create a timer called a **network allocation vector (NAV)** that shows how much time must pass before these stations are allowed to check the channel for idleness. Each time a station accesses the system and sends an RTS frame, other stations start their NAV.

Collision during Handshaking

What happens if there is a collision during the time when RTS or CTS control frames are in transition, often called the handshaking period? Two or more stations may try to send RTS frames at the same time. These control frames may collide. However, because there is no mechanism for collision detection, the sender assumes there has been a collision if it has not received a CTS frame from the receiver. The backoff strategy is employed, and the sender tries again.

Hidden-Station Problem

The solution to the hidden-station problem is the use of the handshake frames (RTS and CTS). Above figure shows that the RTS message from B reaches A, but not C. However, because both B and C are within the range of A, the CTS message, which contains the duration of data transmission from B to A, reaches C. Station C knows that some hidden station is using the channel and refrains from transmitting until that duration is over.

Point Coordination Function (PCF)

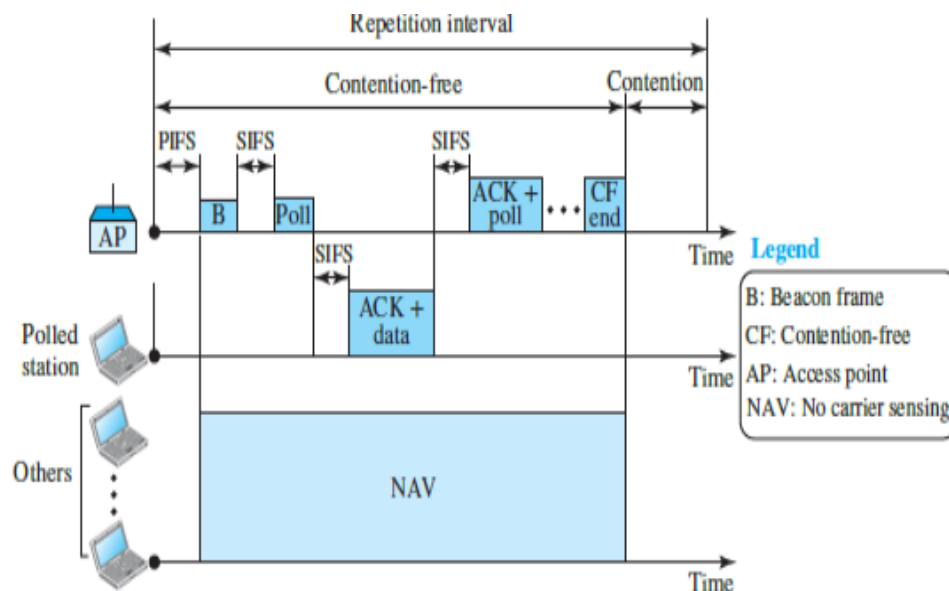
The **point coordination function (PCF)** is an optional access method that can be implemented in an infrastructure network (not in an ad hoc network). It is implemented on

top of the DCF and is used mostly for time-sensitive transmission. PCF has a centralized, contention-free polling access method. The access point performs polling for stations that are capable of being polled. The stations are polled one after another, sending any data they have to the access point.

To give priority to PCF over DCF, another interframe space, has been defined:

point coordination function interframe space [PCF IFS (PIFS)]. PIFS is shorter than the DIFS. This means that if, at the same time, a station wants to use only DCF and an access point wants to use PCF, the access point has priority.

Because of the priority of PCF over DCF, stations that only use DCF may not gain access to the medium. To prevent this, a repetition interval has been designed to cover both contention-free PCF and contention-based DCF traffic. The *repetition interval*, which is repeated continuously, starts with a special control frame, called a **beacon frame**. When the stations hear the beacon frame, they start their NAV for the duration of the contention-free period of the repetition interval. Below figure shows an example of a repetition interval.

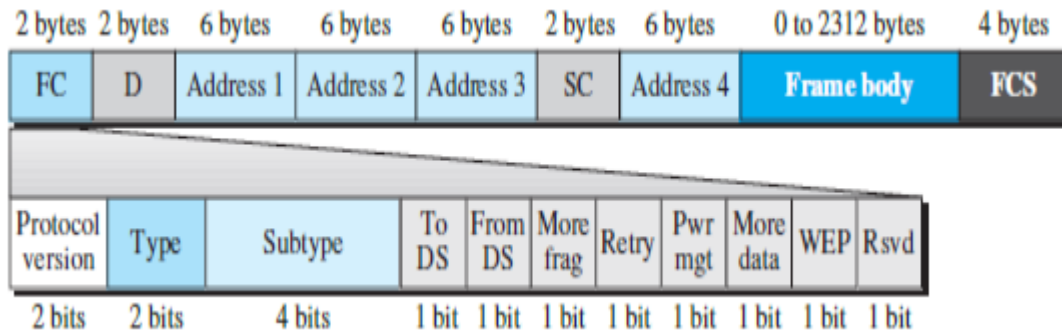


Example of repetition interval

During the repetition interval, the point controller (PC) can send a poll frame, receive data, send an ACK, receive an ACK, or do any combination of these (802.11 uses piggybacking). At the end of the contention-free (CF) period, the PC sends a CF end frame to allow the contention-based stations to use the medium.

Frame Format

The MAC layer frame consists of nine fields, as shown in below Figure.



Frame format

Frame control (FC). The FC field is 2 bytes long and defines the type of frame and some control information. Below table describes the subfields:

Field	Explanation
Version	Current version is 0
Type	Type of information: management (00), control (01), or data (10)
Subtype	Subtype of each type (see Table 4.6)
To DS	Defined later
From DS	Defined later
More flag	When set to 1, means more fragments
Retry	When set to 1, means retransmitted frame
Pwr mgt	When set to 1, means station is in power management mode
More data	When set to 1, means station has more data to send
WEP	Wired equivalent privacy (encryption implemented)
Rsvd	Reserved

D. This field defines the duration of the transmission that is used to set the value of NAV. In one control frame, it defines the ID of the frame.

Addresses. There are four address fields, each 6 bytes long. The meaning of each address field depends on the value of the *To DS* and *From DS* subfields.

Sequence control. This field, often called the SC field, defines a 16-bit value. The first 4 bits define the fragment number; the last 12 bits define the sequence number, which is the same in all fragments.

Frame body. This field, which can be between 0 and 2312 bytes, contains information based on the type and the subtype defined in the FC field.

FCS. The FCS field is 4 bytes long and contains a CRC-32 error-detection sequence.

Frame Types

A wireless LAN defined by IEEE 802.11 has three categories of frames: management frames, control frames, and data frames.

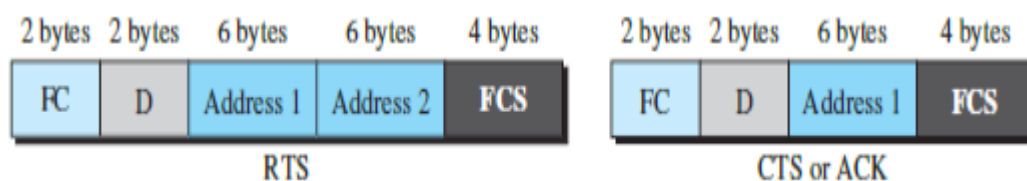
Management Frames

Management frames are used for the initial communication between stations and access Points.

Control Frames

Control frames are used for accessing the channel and acknowledging frames.

Below Figure shows the format.



Control frames

For control frames the value of the type field is 01; the values of the subtype fields for frames are discussed in below table.

<i>Subtype</i>	<i>Meaning</i>
1011	Request to send (RTS)
1100	Clear to send (CTS)
1101	Acknowledgment (ACK)

Data Frames

Data frames are used for carrying data and control information.

Addressing Mechanism

IEEE 802.11 specifies four addressing mechanism cases, defined by the value of the two flags in the FC field, *To DS* and *From DS*. Each flag can be either 0 or 1, resulting in four different situations.

<i>To DS</i>	<i>From DS</i>	<i>Address 1</i>	<i>Address 2</i>	<i>Address 3</i>	<i>Address 4</i>
0	0	Destination	Source	BSS ID	N/A
0	1	Destination	Sending AP	Source	N/A
1	0	Receiving AP	Source	Destination	N/A
1	1	Receiving AP	Sending AP	Destination	Source

Addresses

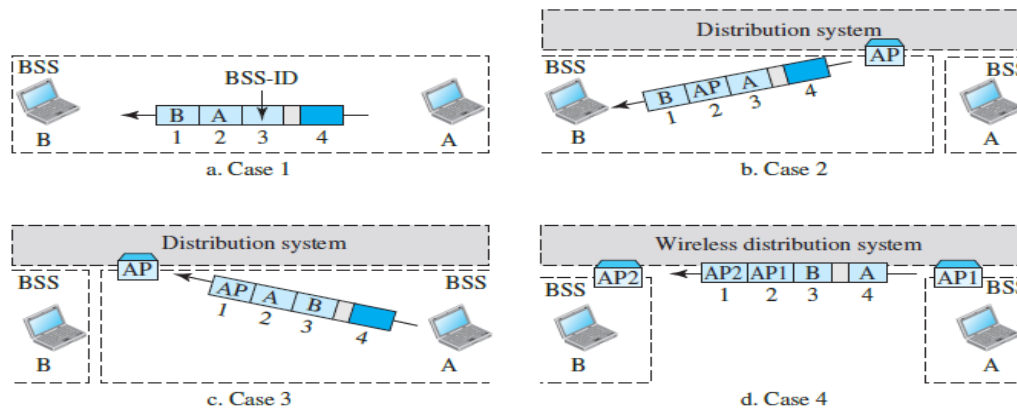
Address 1 is always the address of the next device that the frame will visit. Address 2 is always the address of the previous device that the frame has left. Address 3 is the address of the final destination station if it is not defined by address 1 or the original source station if is not defined by address 2. Address 4 is the original source when the distribution system is also wireless.

Case 1:00. In this case, *To DS* = 0 and *From DS* = 0. This means that the frame is not going to a distribution system (*To DS* = 0) and is not coming from a distribution system (*From DS* = 0). The frame is going from one station in a BSS to another without passing through the distribution system.

Case 2:01. In this case, *To DS* = 0 and *From DS* = 1. This means that the frame is coming from a distribution system (*From DS* = 1). The frame is coming from an AP and going to a station. Note that address contains the original sender of the frame (in another BSS).

Case 3:10. In this case, $To DS = 1$ and $From DS = 0$. This means that the frame is going to a distribution system ($To DS = 1$). The frame is going from a station to an AP. The ACK is sent to the original station. Note that address 3 contains the final destination of the frame in the distribution system.

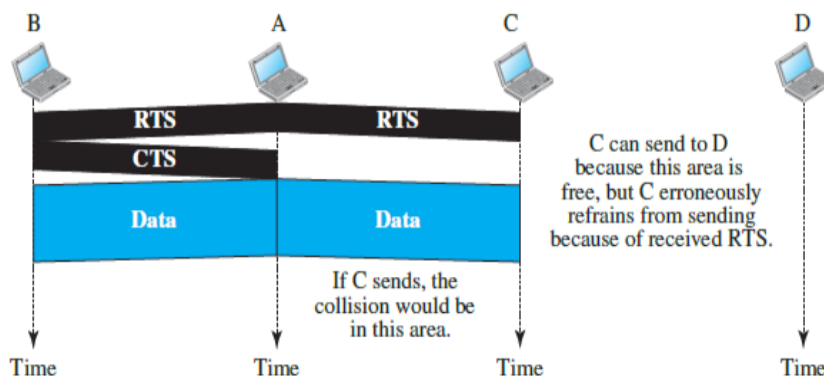
Case 4:11. In this case, $To DS = 1$ and $From DS = 1$. This is the case in which the Distribution system is also wireless. The frame is going from one AP to another AP in a wireless distribution system. Here, we need four addresses to define the original sender, the final destination, and two intermediate. The below figure depicts the cases in detail:



Addressing mechanisms

Exposed-Station Problem

In the *exposed-station problem*, station refrains from using a channel when it is, in fact, available. Station A is transmitting to station B. Station C has some data to send to station D, which can be sent without interfering with the transmission from A to B. However, station C is exposed to transmission from A; it hears what A is sending and thus refrains from sending. In other words, C is too conservative and wastes the capacity of the channel. The handshaking messages RTS and CTS cannot help in this case. Station C hears the RTS from A and refrains from sending, even though the communication between C and D cannot cause a collision in the zone between A and C; station C cannot know that station A's transmission does not affect the zone between C and D. The below figure illustrates the same:



Exposed-station problem